

PROTECTED TAILS AND POLYNOMIAL-TIME ENUMERATION OF PERMUTATIONS AVOIDING A DIRECT SUM OF AN INCREASING PATTERN AND 231

HENNING ULFARSSON

ABSTRACT. We give an exact algorithm counting the permutations that avoid a fixed pattern from the following family: the direct sum of an increasing pattern and the pattern 231. The first members of the family are 1342 and 12453. For each member, the algorithm computes the number of avoiding permutations of every length up to a given bound using polynomially many arithmetic operations and polynomially many stored integers, with degrees that grow linearly in the length of the pattern. We first obtain an exact recurrence by reading a permutation from left to right and recording, at each step, the constraints that the letters read so far impose on those still unread. Its state space grows exponentially, so evaluating it directly takes exponential time. We then show that part of the state is protected: later steps carry it along unchanged and do not depend on it. Factoring the protected part out turns the recurrence into a dynamic program with polynomially many stored transfer entries, and this gives the polynomial bounds for every member of the family. For the pattern 12453, a translation symmetry sharpens the bounds to degree seven for the operations and degree four for the storage. Separately written implementations and exact Chinese-remainder certification determine the number of 12453-avoiding permutations of every length up to 150. The previously published series reached length 38. The same tables also generate uniformly random avoiders in polynomial time. We illustrate this with a heatmap of one million 12453-avoiding permutations of length 300 sampled with floating-point tables. The counting recurrences for 1342 and 12453 are verified in the Lean 4 proof assistant.

1. INTRODUCTION

A permutation $\pi = \pi_1 \cdots \pi_n$ of $[n] = \{1, \dots, n\}$ *contains* a pattern τ in the symmetric group S_k if there are indices $1 \leq i_1 < \cdots < i_k \leq n$ for which $\pi_{i_1}, \dots, \pi_{i_k}$ has the same relative order as τ . Otherwise π *avoids* τ . For example, the entries 2, 5, 7, 9, 6 in 2 5 1 7 9 6 8 3 4 have relative ranks 1, 2, 4, 5, 3, and therefore form an occurrence of 12453. We write $\text{Av}_n(\tau)$ for the set of τ -avoiding permutations of $[n]$, and $\text{Av}(\tau) = \bigcup_{n \geq 0} \text{Av}_n(\tau)$. Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$.

The basic enumerative problem is to determine

$$a_n(\tau) = |\text{Av}_n(\tau)|.$$

Here “polynomial-time enumeration” means computing this integer, not listing the avoiding permutations. This is the algorithmic notion behind Wilfian formulas and systematic methods based on enumeration schemes and insertion encodings [3, 23, 24]. For a fixed pattern, the Marcus–Tardos theorem gives an exponential upper bound $a_n(\tau) \leq c(\tau)^n$ on the counting sequence [16]. The task is to obtain $a_n(\tau)$ in at most $P(n)$ steps, where P is a polynomial. No general principle supplies such an algorithm: Garrabrant and Pak constructed a finitely based class whose exact counting sequence cannot be computed in polynomial time unless $\text{EXP} = \oplus \text{EXP}$ [14].¹

Date: September 2026.

2020 Mathematics Subject Classification. 05A05, 05A15, 68W40.

Key words and phrases. permutation pattern, pattern avoidance, polynomial-time algorithm, Wilfian formula.

¹Here EXP is the class of decision problems solvable in deterministic exponential time, $\oplus \text{EXP}$ is the class of problems that ask whether a nondeterministic exponential-time machine has an odd number of accepting computations, and the two classes are widely believed to differ.

If $\alpha \in S_r$ and $\beta \in S_s$, their *direct sum* $\alpha \oplus \beta$ is the word obtained by writing α and then writing β with every value increased by r . We will focus on the following direct-sum family. Put

$$\iota_d = 12 \cdots d, \quad \beta_d = \iota_d \oplus 231 = 12 \cdots d(d+2)(d+3)(d+1). \quad (1)$$

Thus

$$\beta_1 = 1342, \quad \beta_2 = 12453, \quad \beta_3 = 123564, \quad \beta_4 = 1234675.$$

The case $d = 1$ is already enumerated algebraically by Bóna [8]. The family itself also appears in Bóna's later work: from $\beta_d = 1 \oplus \beta_{d-1}$ for $d \geq 2$, and $\beta_1 = 1 \oplus 231$, he obtained a recursive deletion rule for left-to-right minima (entries smaller than every entry to their left) and used it to determine the Stanley–Wilf limits [9]. That rule does not by itself determine the exact counts from the unrefined β_{d-1} counts. The next case is $d = 2$, giving $\beta_2 = 12453$. The counting sequence begins

$$1, 1, 2, 6, 24, 119, 694, 4581, 33286, 260927, 2174398, 19053058, \dots \quad (2)$$

In the 2022 census, the 12453 sequence was known through $n = 38$ from a specialized program of Biers-Ariel [6, 7, 11]. The program uses a composition-valued state of unbounded length, and the number of its states grows exponentially. Our starting point is the same recurrence: for $d = 2$ the literal recurrence developed below coincides with Biers-Ariel's, state for state, and we extend it to the whole family β_d . We then show how to shrink its exponential state space to polynomial size.

The family also has a description by a sorting device. Albert, Homberger, Pantone, Shar, and Vatter [2] study the $\mathcal{C}$ -machine, which pushes each input entry anywhere into a container whose contents must remain in the class $\mathcal{C}$ and pops the container from its left end. Their Theorem 1.1 shows that the $\text{Av}(B)$ -machine generates exactly $\text{Av}(1 \ominus B)$, and they observe that $\text{Av}(4213)$, the complement of $\text{Av}(1342)$, is generated by the $\text{Av}(213)$ -machine, answering a question of Bóna. Since $\beta_d^c = 1 \ominus \beta_{d-1}^c$ for the complement β_d^c , the class $\text{Av}(\beta_d)$ is, up to complementation, the class generated by the $\text{Av}(\beta_{d-1}^c)$ -machine. For 12453 the container class is $\text{Av}(4213)$. They prove that the generated class has a rational generating function when $\mathcal{C}$ is finite, an algebraic one when $\mathcal{C}$ is bounded, and a Wilfian formula, that is, a polynomial-time counting algorithm [24], when $\mathcal{C}$ is a polynomial class. The container class $\text{Av}(4213)$ has growth rate 8 and is none of these, so their sufficient conditions do not apply to 12453. The protected-tail factorization below supplies the Wilfian formula outside them.

In a permutation that avoids β_d , an increasing subsequence of length d creates a condition on a future 231-avoiding projection, and the conjunction of the conditions created by a prefix is represented by an ordered stack of intervals. A transition changes the control (the counts of unread values that no obligation created so far restricts), the active head of the stack, or both. The deferred tail remains unchanged until every interval preceding it has been consumed, and the transitions before that time do not depend on the tail. A transfer kernel summarizes this part of the computation by recording the control at which the tail becomes available. This factorization removes the stack from the memoization key, and the resulting transfer table has polynomial size.

Our main result is the following. The notation $O_d(\cdot)$ allows the hidden constant to depend on the fixed family parameter d .

Theorem 1.1. *For every fixed $d \geq 1$ and every $N \geq 0$, the numbers*

$$|\text{Av}_0(\beta_d)|, \dots, |\text{Av}_N(\beta_d)|$$

can be computed using $O_d(N^{3d+2})$ exact-integer arithmetic operations and $O_d(N^{2d+1})$ stored integers. All integers involved have $O_d(N \log N)$ bits. Consequently the bit complexity² is polynomial.

²The operation count treats each integer addition or multiplication as one step. The bit complexity counts operations on single bits, so the sizes of the integers enter through the multiplication cost $M(b)$ defined below.

More precisely it is

$$O_d\left(N^{3d+2}M(N \log N)\right),$$

where $M(b)$ is the cost of multiplying b -bit integers.

In particular, the family theorem gives time $O(N^5)$ and storage $O(N^3)$ for $\text{Av}(1342)$, and $O(N^8)$ and $O(N^5)$ for $\text{Av}(12453)$. An additional translation symmetry, proved in [section 7](#), improves this to $O(N^7)$ and $O(N^4)$ for $\text{Av}(12453)$ ([corollary 7.4](#)). The theorem is polynomial for each fixed d . It is not a uniform polynomial bound when d is part of the input.

To our knowledge, no exact polynomial-time enumeration algorithm was previously available for the family $\text{Av}(\iota_d \oplus 231)$ at every fixed d , or even for the single class $\text{Av}(12453)$. The contributions are threefold. First, we show that the literal recurrence is a pushdown system: every transition acts on the interval stack through its active head alone, so the completion counts factor through transfer kernels ([theorem 2.17](#) and [corollary 5.1](#)) despite Fibonacci-many reachable literal stack states ([proposition 4.5](#)). The factorization itself is the counting form of the classical conversion of a pushdown automaton into a grammar ([remark 2.18](#)). What is new is that it applies with a control set that grows with n , where it yields a transfer table of polynomial size rather than a finite grammar. Second, we prove explicit family bounds ([theorem 1.1](#)) and two translation quotients for 12453 ([lemmas 7.2](#) and [7.3](#)). Third, we provide separately written implementations and a Chinese-remainder certification of the values $|\text{Av}_n(12453)|$ for $n \leq 150$ ([proposition 8.1](#)). In addition, the transfer tables give uniform random generation of avoiders in polynomial time ([proposition 8.2](#)), and the counting recurrences for $d \leq 2$ have been machine-checked in Lean 4 ([section 9](#)).

2. TRIGGERS, STACKS, AND THE COMPLETE ONE-THRESHOLD PROTOTYPE

We first describe the restrictions that a scanned prefix imposes on its completions and show that they admit an interval-stack representation for every $d \geq 1$. This part ([lemmas 2.1, 2.2, 2.5](#) and [2.8](#) and [proposition 2.10](#)) is reused unchanged for the general family in [section 4](#), and [theorem 2.17](#) is stated for recurrences on a control set and a stack, so that it applies verbatim in [section 5](#). We then develop the counting algorithm completely for $d = 1$. This case isolates the protected-tail construction before the additional threshold coordinates are introduced.

For a word w with distinct entries, its *standardization* $\text{st}(w)$ is obtained by replacing the smallest entry by 1, the next smallest by 2, and so on. If X is a set of values, $w|_X$ denotes the subword formed by retaining only the entries belonging to X . An *increasing d -subsequence* of a permutation π , or more generally of a word π with distinct entries, is a choice of positions $i_1 < \dots < i_d$ with $\pi_{i_1} < \dots < \pi_{i_d}$. We call its last entry a *d -trigger*. The positions need not be adjacent.

Lemma 2.1 (Trigger lemma). *A permutation $\pi = \pi_1 \dots \pi_n$ avoids β_d if and only if, for every increasing d -subsequence ending at $c = \pi_j$, the word*

$$\pi_{j+1} \dots \pi_n|_{\{x: x > \pi_j\}}$$

avoids 231 after standardization.

Proof. The first d entries of an occurrence of β_d form an increasing d -subsequence, and its last three entries form a 231 whose values are all larger. Conversely, an increasing d -subsequence followed by such a 231 has relative order $\iota_d \oplus 231 = \beta_d$. $\square$

When a trigger c is read, its entire subsequent projection onto values larger than c must avoid 231. Later in the scan, some letters of that projection have already been read. Merely testing the still-unread projection would lose mixed occurrences involving both the read and unread portions. The interval stack below records the resulting conditions on the unread letters.

The 231-obligation created by a trigger $c = \pi_j$ is the requirement that $\pi_{j+1} \cdots \pi_n|_{\{x:x>c\}}$ avoid 231 after standardization. For a trigger c in a scanned prefix, its *residual obligation* is the restriction of this requirement to completions of that prefix, with the already scanned letters of the projection fixed: if $\sigma = \sigma_1 \cdots \sigma_k$ is the prefix, $c = \sigma_j$, the letters of $[n]$ not in σ are *unread*, and w is a *completion* of σ , that is, a word using each unread letter once, then w satisfies the residual obligation of c when $(\sigma_{j+1} \cdots \sigma_k w)|_{\{x:x>c\}}$ avoids 231 after standardization.

2.1. The Catalan interval stack. The following first-letter decomposition of $\text{Av}(231)$ applies to an arbitrary finite ordered alphabet.

Lemma 2.2 (First-letter lemma). *Let I be a finite totally ordered set of size ℓ , and let $x \in I$ have local rank $r = |\{y \in I : y \leq x\}|$. A permutation of I beginning with x avoids 231 if and only if all values below x occur before all values above x , and the two induced subwords avoid 231. Thus reading x replaces an interval of size ℓ by intervals of sizes*

$$r - 1, \quad \ell - r,$$

in that order, with zero parts deleted.

Proof. If an upper value occurs before a lower value after x , these three entries have pattern 231. Conversely, if the lower and upper subwords occur in that order, a 231 cannot use letters from both: both position and value increase from the first block to the second. Nor can a 231 use x and two letters from just one block: x is larger than every lower letter and smaller than every upper letter, whereas the first entry of a 231 has middle rank. If it uses x , one lower letter, and one upper letter, their order is 213. Avoidance therefore reduces to avoidance inside the two blocks. $\square$

Write $\text{Av}(231)(I)$ for the 231-avoiding words using each value of I once. If $I < J$, that is, every element of I is smaller than every element of J , and u and w are words on I and J , then $u \oplus w$ is the concatenation uw . For sets of such words, $\oplus$ denotes the set of all concatenations. Consider the projection created by one trigger c , and read its letters one at a time. By lemma 2.2, the first letter x splits the rest of the projection into a lower and an upper interval, and every letter of the lower interval must be read before any letter of the upper one. Hence the next letter of the projection lies in the lower interval while that interval is nonempty, and the lemma, applied to the subword on that interval, splits it in turn. The letter itself then plays no further role, since both conditions of the lemma concern only the letters after it. Repeated application of lemma 2.2 thus represents the residual obligation of c by an ordered interval stack

$$I_1 < I_2 < \cdots < I_s,$$

that is, the unread letters of the projection must form a word in

$$\text{Av}(231)(I_1) \oplus \text{Av}(231)(I_2) \oplus \cdots \oplus \text{Av}(231)(I_s). \quad (3)$$

The read letters of the projection do not appear in (3), and none of them is needed again: an occurrence of 231 involving a read letter x and an unread letter must begin with x , because if a letter $a < x$ read earlier were its first entry, every value of the projection below a would already have been read, since the lower interval of a precedes its upper one, which contains x . Beginning with x , the occurrence needs an upper letter followed by a lower one, which the split at x excludes. For instance, relabel the values of a projection as $1, \dots, 6$ and suppose that its first two letters are 4 and 2. Reading 4 leaves the stack $\{1, 2, 3\} \mid \{5, 6\}$, and reading 2 then leaves $\{1\} \mid \{3\} \mid \{5, 6\}$. The completion 1, 3, 6, 5 is allowed, whereas a completion beginning 3, 1 is excluded: together with the read letter 2 it would form the occurrence 2, 3, 1 of 231.

The stack records the obligations created by the scanned prefix. Further obligations arise as the completion is read, and they are incorporated by the stack updates described below. Here

and throughout, an *interval* is a consecutive block of the ordered set of currently unread values. Its elements need not be consecutive integers. We call I_1 the *active head* and $(I_2, \dots, I_s)$ the *deferred tail*. Only the active head may be read next among the values of the stack: every letter of I_1 must be read before any letter of I_2 , and so forth. Unread values below the trigger are constrained by no obligation created so far and may be interspersed. For an interval of size ℓ , an *endpoint choice* has local rank 1 or ℓ , and an *interior choice* has local rank in $\{2, \dots, \ell - 1\}$.

We call the gap in value between two consecutive intervals of the current stack a *separation*, and a read letter lying in that gap a *separator*. A separation may have several separators. An interior choice creates a new separation, at the letter read, while an endpoint choice leaves at least one side empty and creates none, although the letter read may lie in an existing separation. Inductively, as long as no trigger smaller than c has been read (a trigger larger than c is itself a letter of the projection and only splits it further), two unread letters $u < v$ of the projection lie in different intervals if and only if some letter y with $u < y < v$ has been read from the projection. [Lemma 2.11](#) and [corollary 4.4](#) below determine the separations of the stack once the obligations of several triggers have been combined.

Example 2.3 (Reading one projection). The permutation

$$\pi = 9, 11, 10, 14, 5, 12, 6, 2, 3, 8, 4, 7, 1, 13, 15 \quad (4)$$

of [15] avoids 1342, hence also 12453, and serves as the running example of this article. For $d = 1$ every letter is a trigger. The projection created by the first letter 9 is the word 11, 10, 14, 12, 13, 15 formed by the later letters above 9. It standardizes to 215346 and avoids 231, as [lemma 2.1](#) requires. We read this projection letter by letter. The stack starts as the single interval $\{10, \dots, 15\}$. The first letter 11 has local rank 2, an interior choice, so [lemma 2.2](#) requires the lower interval $\{10\}$ to be read before the upper interval $\{12, 13, 14, 15\}$. The stack is $\{10\} \mid \{12, \dots, 15\}$. Reading 10 is an endpoint choice that exhausts the head and leaves $\{12, \dots, 15\}$, and the interior choice 14, of local rank 3 in that interval, splits it into $\{12, 13\} \mid \{15\}$. The unread letters of the projection must now form a word in $\text{Av}(231)(\{12, 13\}) \oplus \text{Av}(231)(\{15\})$: either order of 12 and 13, then 15. The read letters 11, 10, 14 are not retained. While 10 was unread, 11 was the only separator of the separation between $\{10\}$ and $\{12, \dots, 15\}$. Now 14 is the only separator of the separation between $\{12, 13\}$ and $\{15\}$, and 10 and 11 separate nothing.

Several triggers impose overlapping obligations. The following definition fixes the object that records all of them: a read prefix together with a stack, updated by three kinds of move. [Lemma 2.8](#) and [proposition 2.10](#) then show that the stack of a legal prefix represents exactly the conjunction of the residual obligations of its triggers, and that the prefixes that can be read are exactly the prefixes of β_d -avoiding permutations.

Definition 2.4 (Scan states and legal moves). Fix $d \geq 1$ and n . A *scan state* is a pair (σ, S) , where σ , the *read prefix*, is a word with distinct letters from $[n]$, and $S = (I_1, \dots, I_s)$, the *stack*, is a list of nonempty sets with $I_1 < I_2 < \dots < I_s$ whose union is the set of unread letters larger than the least d -trigger q of σ , and is empty when σ has no d -trigger. The *initial state* is $(\varepsilon, \emptyset)$: the empty prefix and the empty stack, $s = 0$. Let x be an unread letter. Reading x is a *legal move*, leading to the state $(\sigma x, S')$, in exactly the following three cases.

- (a) *Merger*: x is a d -trigger of σx and is smaller than every d -trigger of σ . Let E be the set of unread letters strictly between x and q , or the set of all unread letters larger than x when σ has no d -trigger. Then $S' = (E \cup I_1, I_2, \dots, I_s)$ if $s \geq 1$, and $S' = (E)$ if $s = 0$, the empty stack when $E = \emptyset$.
- (b) *Split*: x is a d -trigger of σx that exceeds some d -trigger of σ , and $x \in I_1$. Then $S' = (I_1^{<x}, I_1^{>x}, I_2, \dots, I_s)$, where $I_1^{<x}$ and $I_1^{>x}$ are the sets of elements of I_1 below and

above x , empty sets being deleted. The move is an endpoint or an interior choice according to the local rank of x in I_1 , as defined above.

- (c) *Non-trigger move*: x is not a d -trigger of σx . Then $S' = S$. For $d = 1$ every letter is a 1-trigger, so this case does not occur.

Reading any other unread letter is *illegal*. A word σ is *legal* if the initial state reaches a state (σ, S) by legal moves. Then S is determined by σ , and we write $\mathcal{S}(\sigma)$ for it and call it the stack of σ . Finally, $\mathcal{R}(\sigma)$ denotes the set of permutations of $[n]$ with prefix σ all of whose prefixes are legal, the permutations that *can be read* from σ .

It is useful to note that, if σ has a d -trigger, then every unread letter $x > q$ is a d -trigger of σx exceeding q , since replacing the last entry of an increasing d -subsequence ending at q by x gives one ending at x . For $d = 1$ a legal move is a new left-to-right minimum, which merges, or a letter of the active head, which splits. [Figure 1](#) illustrates a merger.

Lemma 2.5 (Legal moves). *Let (σ, S) be a scan state and let x be an unread letter.*

- (a) *If reading x is a legal move, then $(\sigma x, S')$, with S' as in [definition 2.4](#), is a scan state.*
- (b) *Reading x is illegal if and only if x lies in a deferred interval $I_2, \dots, I_s$.*

Proof. (a) In a merger, x becomes the least d -trigger of σx . If σ has no d -trigger, then S is empty and E is the set of all unread letters above x , so $S' = (E)$ is a scan state with σx . Otherwise the unread letters of σx larger than x are those of E together with those larger than q , that is, $E \cup I_1 \cup \dots \cup I_s$, the union of S' . Every letter of E is smaller than q and every letter of S is larger, so $E < I_1$ and the sets of S' are disjoint and increasing. In a split, the least d -trigger is still q , since $x > q$, and the unread letters above q lose exactly x , which is what the split removes. The two parts of I_1 lie below and above x . In a non-trigger move, $x < q$ if σ has a d -trigger, since an unread $x > q$ is a d -trigger of σx as noted above, so the least d -trigger and the unread letters above it are unchanged, and so is S . If σ has no d -trigger, then S is empty and a non-trigger move leaves it empty.

(b) Let x lie in a deferred interval. Then $x > q$, so x is a d -trigger of σx exceeding q , as noted above. Hence cases (a) and (c) of [definition 2.4](#) do not apply, and $x \notin I_1$ excludes case (b). Conversely, let x be illegal. Since case (c) does not apply, x is a d -trigger of σx . Since case (a) does not apply, x exceeds some d -trigger of σ , so $x > q$. Hence x lies in the stack, and in a deferred interval since case (b) does not apply. $\square$

Definition 2.6 (Faithful stacks). Let (σ, S) be a scan state with $S = (I_1, \dots, I_s)$. We say that S is *faithful to σ* if a completion w of σ satisfies the residual obligations of all d -triggers of σ precisely when

$$w|_{I_1 \cup \dots \cup I_s} \in \text{Av}(231)(I_1) \oplus \dots \oplus \text{Av}(231)(I_s),$$

where the right-hand side is the empty word when $s = 0$.

A scan state fixes the shape of the stack. Faithfulness is the semantic property that the stack describes exactly the obligations created so far, and a stack may or may not have it. [Lemma 2.8](#) shows that a merger preserves faithfulness, and [proposition 2.10](#) that the stack $\mathcal{S}(\sigma)$ of a legal prefix is faithful to σ .

Example 2.7 (Faithful and unfaithful stacks). Let $d = 1$ and let $\sigma = 9, 11, 10, 14$ be the prefix of the running example (4) scanned in [example 2.3](#). Its least trigger is 9, and the unread letters above 9 are 12, 13, 15, so the stacks $(\{12, 13\}, \{15\})$, $(\{12, 13, 15\})$ and $(\{12\}, \{13\}, \{15\})$ all form scan states with σ . For $d = 1$ every letter of σ is a trigger, and a completion w satisfies their residual obligations exactly when 11, 10, 14, $w|_{\{y:y>9\}}$ and 14, $w|_{\{y:y>11\}}$ avoid 231, the obligations of 10 and 14 adding nothing further. As computed in [example 2.3](#), this holds exactly when w reads 12 and 13, in either order, before 15: a completion reading 15 first completes the occurrence

14, 15, x of 231 with $x \in \{12, 13\}$. Hence $(\{12, 13\}, \{15\})$, the stack produced by the legal moves, is faithful to σ . The stack $(\{12, 13, 15\})$ is not: its language contains the order 15, 12, 13, which violates the obligation of 9. Nor is $(\{12\}, \{13\}, \{15\})$: its language excludes the order 13, 12, 15, which satisfies every obligation.

Lemma 2.8 (Stack merger). *Let (σ, S) be a scan state whose stack $S = (I_1, \dots, I_s)$ is faithful to σ , and let x be an unread letter that is a d -trigger of σx and is smaller than every d -trigger of σ .*

- (a) *If σ has no d -trigger, then S is empty, and the stack (E) , where E is the set of unread letters larger than x , is faithful to σx ; it is the empty stack when $E = \emptyset$.*
- (b) *If σ has a d -trigger, let q be the least one and let E be the set of unread letters strictly between x and q . Then E is empty or an interval of the unread alphabet; if $s \geq 1$, then $E < I_1$ and $E \cup I_1$ is an interval of the unread alphabet; and the merged stack*

$$(E \cup I_1, I_2, \dots, I_s),$$

or (E) when $s = 0$, with an empty set deleted, is faithful to σx . In particular only the active head changes. The old tail $I_2, \dots, I_s$ is unchanged.

Proof. We use repeatedly that a direct sum of 231-avoiding words avoids 231, since positions and values both increase from each block to the next. Let w be a completion of σx . Then xw is a completion of σ , and since x is smaller than every d -trigger of σ , the words w and xw satisfy the same residual obligations of the d -triggers of σ . Moreover x lies in no stack interval: if σ has no

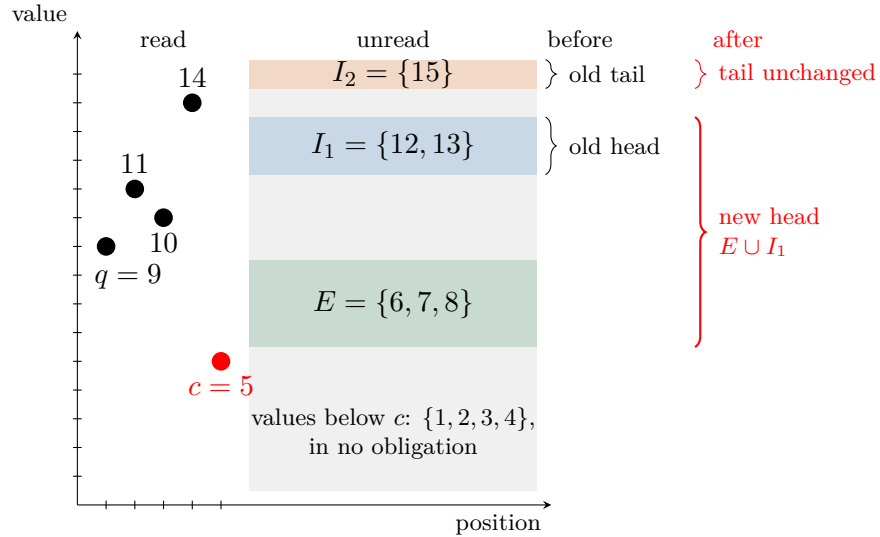


FIGURE 1. The stack merger of lemma 2.8 for $d = 1$ after the prefix 9, 11, 10, 14 of the running example (4), as in example 2.9. The triggers 9, 11, 10, 14 have been read: 11 split the projection of 9, 10 exhausted the lower part, and 14 split again, leaving the head $I_1 = \{12, 13\}$ and the tail $I_2 = \{15\}$. The new trigger $c = 5$ is smaller than every earlier trigger, so $q = 9$ and $E = \{6, 7, 8\}$ is the set of unread values strictly between c and q . The obligation of c merges E into the head: the stack becomes $\{6, 7, 8, 12, 13\} \mid \{15\}$, and the tail is unchanged. The read letters 9, 10, 11 lie in value between E and I_1 but create no separation: no letter smaller than 8 had been read when they were read, and no earlier obligation involves E , every earlier trigger being at least q . The letter 14, read after the trigger $9 < 13$, does separate the head from the tail, and that separation survives the merger.

d -trigger, the stack is empty by [definition 2.4](#), and otherwise the stack consists of unread letters larger than the least d -trigger q of σ , while $x < q$. Hence xw and w have the same restriction to the stack values. The residual obligations of the d -triggers of σx are those of σ together with the obligation of x , which requires $w|_{\{y:y>x\}}$ to avoid 231. Moreover, x is the least d -trigger of σx .

(a) If σ has no d -trigger, then S is empty by [definition 2.4](#), and x is the only d -trigger of σx . The unread letters larger than x form E , and the residual obligations of σx reduce to the obligation of x , that $w|_E$ avoid 231. Hence (E) is faithful to σx .

(b) By [lemma 2.5\(a\)](#), the merged stack together with σx is a scan state, and its union is the set of unread letters above x . Since $I_1 \cup \dots \cup I_s$ is the whole set of unread letters above q and $I_1 < \dots < I_s$, each I_j is an interval of the unread alphabet. Any unread value between two values of E lies strictly between x and q , hence in E , so E is an interval too. Every value of E is smaller than q , while every stack value is larger than q . Hence $E < I_1$ when $s \geq 1$. No unread value lies strictly between E and I_1 : it would lie either strictly between x and q , hence in E , or above q , hence in the stack. Hence $E \cup I_1$ is an interval of the unread alphabet. Reading x , which lies below E , does not change this.

It remains to prove faithfulness: that w satisfies the residual obligations of σx exactly when its restriction to the merged stack lies in the corresponding direct sum. If $s = 0$, faithfulness of the empty stack to σ says that every completion of σ satisfies the residual obligations of all d -triggers of σ , and no unread value exceeds q . Hence the condition on w is the obligation of x on E alone, as in (a). Assume $s \geq 1$. If $E = \emptyset$, the obligation of x asks for 231-avoidance on $I_1 \cup \dots \cup I_s$, which the old condition already implies by the direct-sum fact. Hence the old stack is unchanged and is still faithful to σx . Assume $E \neq \emptyset$. If $s = 1$, the obligation of x is 231-avoidance on $E \cup I_1$, which is the claimed condition, and it implies the old condition on the subword I_1 . Assume $s \geq 2$.

Necessity. Suppose w satisfies the residual obligations of σx . The old condition places every value of I_1 before every value of I_2 , and hence before every later interval, and makes $w|_{I_j}$ avoid 231 for every j . The obligation of x makes $w|_{E \cup I_1}$ avoid 231. If some $z \in I_2$ occurred before some $e \in E$, choose any $y \in I_1$: the old ordering puts y before z , so $y, z, e \sim 231$ in $w|_{\{y:y>x\}}$, contradicting the obligation of x . The same argument applies to every later interval, so all of $E \cup I_1$ precedes the old tail, and the restriction of w to the merged stack lies in the merged direct sum.

Sufficiency. Suppose the restriction of w to the merged stack lies in the merged direct sum. Its restriction to $I_1 \cup \dots \cup I_s$ then lies in the old direct sum, so w satisfies the residual obligations of the d -triggers of σ . Moreover, $w|_{\{y:y>x\}}$ is a direct sum of 231-avoiding words, so it avoids 231, which is the obligation of x . $\square$

The merger creates no separation between E and I_1 , although read letters lie between them in value: the old least trigger q and the earlier triggers above q , among them letters read from earlier active heads. When $d \geq 2$, earlier letters that were not triggers may lie inside the value range of E as well, and they create no separation either. All of these letters were read before the new trigger, so none belongs to its projection, and no earlier obligation involves E , since every earlier trigger is at least q . A separation produced by a split is therefore not permanent: once the interval below it has been exhausted, a later merger can extend the head past the letters that produced it, as rows 2–5 of [example 2.12](#) show. [Lemma 2.11](#) and [corollary 4.4](#) determine the separations of the stack exactly.

Example 2.9 (One merger). Continue [example 2.3](#). The fifth letter 5 of π is a trigger smaller than the earlier triggers 9, 11, 10, 14. In [lemma 2.8\(b\)](#), $q = 9$ and $E = \{6, 7, 8\}$, the unread values between 5 and 9. The current stack $\{12, 13\} \mid \{15\}$ consists exactly of the unread values above 9, and the stack is faithful to the prefix by [proposition 2.10\(a\)](#) below, since 9, 11, 10, 14 is a legal

prefix. The merged stack is

$$\{6, 7, 8, 12, 13\} \mid \{15\}.$$

The values 6, 7, 8 join the head without a separation: the read letters 9, 10, 11, which lie between them and 12 in value, were read before the trigger 5 and constrain nothing in E . They must, however, precede 15: if 15 came before 6, then either the old obligation is already violated, or 12 precedes 15 and $12, 15, 6 \sim 231$ in the projection of 5, that is, $5, 12, 15, 6 \sim 1342$. Inside the head only 231-avoidance remains. The permutation π continues with 12 before 6, and 13, 12, 6, 7, 8 would have been allowed as well. For $d = 2$ the same permutation is scanned in [example 4.2](#): there the first 2-trigger is 11 rather than 9, and the corresponding merger occurs at the 2-trigger 6, with $q = 10$ and $E = \{7, 8\}$.

Proposition 2.10 (Scan states of avoiders). *Let σ be a legal word with stack $\mathcal{S}(\sigma) = (I_1, \dots, I_s)$, and let q be the least d -trigger of σ when σ has one. The pair $(\sigma, \mathcal{S}(\sigma))$ is a scan state, by induction along the legal moves from the initial state using [lemma 2.5\(a\)](#).*

- (a) *The stack $\mathcal{S}(\sigma)$ is faithful to σ : a completion w of σ satisfies the residual obligations of all d -triggers of σ if and only if*

$$w|_{I_1 \cup \dots \cup I_s} \in \text{Av}(231)(I_1) \oplus \dots \oplus \text{Av}(231)(I_s),$$

where the right-hand side is the empty word when $s = 0$.

- (b) *If reading the unread letter x from σ is illegal, then no β_d -avoiding permutation has the prefix σx .*
 (c) *$\mathcal{R}(\sigma)$ is the set of β_d -avoiding permutations of $[n]$ with prefix σ . In particular $\mathcal{R}(\varepsilon) = \text{Av}_n(\beta_d)$, and a word is legal if and only if it is a prefix of a β_d -avoiding permutation.*

Proof. We prove (a) by induction along σ . For the empty word there is no trigger, no obligation and no stack, so the empty stack is faithful. Let $S = \mathcal{S}(\sigma)$ be faithful to σ , let x be a legal move, and consider σx in the three cases of [definition 2.4](#).

In a merger, x is a d -trigger of σx smaller than every d -trigger of σ , and by the induction hypothesis S is faithful to σ . The stack S' of [definition 2.4\(a\)](#) is the stack of [lemma 2.8\(a\)](#) or (b) according as σ has no d -trigger or has one, so S' is faithful to σx .

In a split, $x \in I_1$, so $x > q$ and x is a d -trigger of σx as noted after [definition 2.4](#). A completion w of σx satisfies the residual obligations of the triggers of σ if and only if the completion xw of σ does, that is, by the induction hypothesis, if and only if $(xw)|_{I_1} \in \text{Av}(231)(I_1)$, $w|_{I_j} \in \text{Av}(231)(I_j)$ for $j \geq 2$, and w reads the sets $I_1 \setminus \{x\}, I_2, \dots, I_s$ in this order. By [lemma 2.2](#), the first condition holds if and only if $w|_{I_1 \setminus \{x\}} \in \text{Av}(231)(I_1^{<x}) \oplus \text{Av}(231)(I_1^{>x})$. Together, these conditions say that the restriction of w to the stack values lies in the direct sum of the split stack. The obligation of the new trigger x adds nothing: its projection consists of the unread letters above x , namely $I_1^{>x} \cup I_2 \cup \dots \cup I_s$, on which the split stack already prescribes a direct sum of 231-avoiding words, and such a word avoids 231 because positions and values both increase from each block to the next.

In a non-trigger move, x is not a d -trigger of σx . If σ has no d -trigger, there is no obligation and no stack before or after the move. Otherwise $x < q$, as in the proof of [lemma 2.5](#), so x belongs to no projection of a d -trigger of σ , every projection consisting of letters larger than its trigger, which is at least q . The residual obligations and the stack are therefore unchanged, and S remains faithful to σx .

For (b), let x be illegal. By [lemma 2.5\(b\)](#), x lies in a deferred interval I_j , $j \geq 2$. Every completion of σ beginning with x reads x before the letters of the nonempty set I_1 , so its restriction to the stack values is not in the direct sum of (a), and by (a) it violates the residual obligation of some d -trigger c of σ : the projection created by c contains 231, and the permutation contains β_d by [lemma 2.1](#).

For (c), let π be a permutation with prefix σ . If π avoids β_d , then every prefix of π is legal: the empty prefix is, and if a prefix σ' of π is legal and the next letter x of π were illegal, (b) would forbid the prefix $\sigma'x$ of the avoider π . Hence $\pi \in \mathcal{R}(\sigma)$. Conversely, let $\pi \in \mathcal{R}(\sigma)$. Then π itself is legal, no letter is unread, the stack is empty, and by (a) the empty completion satisfies the residual obligation of every d -trigger of π . That is, the projection of every d -trigger avoids 231 after standardization, and π avoids β_d by [lemma 2.1](#). This proves the first sentence of (c) and, with $\sigma = \varepsilon$, that $\mathcal{R}(\varepsilon) = \text{Av}_n(\beta_d)$. A prefix of an avoider is legal by the first part. Conversely, a legal σ has an avoiding completion: read the unread letters smaller than q , or all unread letters when σ has no d -trigger, in decreasing order, and then the intervals $I_1, \dots, I_s$ in order, each in decreasing order. Each letter of the first group is the largest unread letter below the least trigger, or the largest unread letter when there is none, so it is either not a d -trigger or a new least d -trigger with $E = \emptyset$. In both cases the move is legal and the stack is unchanged. Each later letter is the greatest element of the active head, a d -trigger exceeding q as noted after [definition 2.4](#), so reading it is an endpoint choice. All moves are legal, so the completed permutation lies in $\mathcal{R}(\sigma)$ and avoids β_d . $\square$

2.2. The one-threshold state. For $d = 1$, every entry is a trigger, so the least d -trigger q of [definition 2.4](#) is the least letter m read so far. Before the first letter, m is a virtual value larger than every letter. A legal move is a new left-to-right minimum, which adjoins the unread values between it and m to the active head or creates the stack when the stack is empty, or a letter of the active head, which shortens the head at an endpoint choice and replaces it by its lower and upper subintervals at an interior choice. By [definition 2.4](#) and [lemma 2.5](#), the stack of a legal prefix consists of the unread letters above m , and by [proposition 2.10\(c\)](#) the legal prefixes are the prefixes of 1342-avoiding permutations. The following lemma determines the separations of the stack. Two unread values are *adjacent* if no unread value lies strictly between them. By a *prefix* of a 1342-avoiding permutation we mean an initial segment of one. This is stronger than avoiding 1342, since 134 avoids 1342 but, as a prefix of a permutation of $[4]$, its only completion is 1342.

Lemma 2.11 (Separations of the one-threshold stack). *After a legal prefix has been read (equivalently, by [proposition 2.10\(c\)](#), a prefix of a 1342-avoiding permutation), let $u < v$ be adjacent unread letters above m . Then u and v lie in different intervals of the stack if and only if some letter x with $u < x < v$ was read after a letter smaller than u , that is, at a time when the minimum read so far was already below u .*

Proof. We induct along the scan. For the empty prefix there is nothing to prove. The next legal letter is a new minimum or a letter of the active head. Suppose first that the next letter x is a new minimum, adjoining E to the head or creating the stack (E). An adjacent pair $u < v$ above the new minimum that was not an adjacent pair above the old minimum has $u \in E$, so u is smaller than the old minimum and hence smaller than every letter read before x . Every letter between u and v was read before x , so no letter between them was read after a letter smaller than u . Moreover v lies in E or is the least element of I_1 , and $E \cup I_1$, or E when the old stack was empty, is a single interval of the updated stack. Thus neither condition holds for such a pair. Every other adjacent pair lies above x and keeps its members, its intervals, and the letters between them, so both conditions are unchanged for it. Suppose instead that x is read from the active head I_1 . Only the unread neighbours $u < x < v$ of x can form a new adjacent pair, and only when both exist and lie above m . This excludes $x = \min I_1$, where u lies below m . For an interior choice, u and v lie in the two parts created by the split. For $x = \max I_1$ with $|I_1| \geq 2$, u lies in the shortened head and v is the least element of I_2 if a deferred interval remains. Otherwise x is the largest unread letter and no new pair arises. In every case u and v lie in different intervals, and x , read after $m < u$, witnesses the second condition. Every other adjacent pair, and every letter between its members, is unchanged. $\square$

Hence the stack is recovered from the scanned prefix alone: its values are the unread letters above m , and its intervals are the blocks into which this increasing list is cut at exactly the adjacent pairs of the lemma. The precedence recorded by a separation is also visible directly: if x was read after $c < u$ and v preceded u , then c, x, v, u would form 1342. Conversely, two letters of the same interval can occur in either order in a 1342-avoiding completion. [Corollary 4.4](#) proves this and extends both statements to every d . Adjacency cannot be dropped: after the prefix 3, 5, 1 of a 1342-avoiding permutation of $[7]$ the stack is $\{2, 4\} \mid \{6, 7\}$, so 2 and 6 lie in different intervals, although no letter between them was read after a letter smaller than 2.

Example 2.12 (A one-threshold scan). We scan the permutation π of (4) completely. The table lists, after each letter π_i , the type of move, the current minimum m , the interval stack, and the symbol $W_p(L)$ of [section 2.3](#), where p is the number of unread letters below m , L lists the interval sizes, and $W_p(L)$ is the number of 1342-avoiding completions of the prefix. Every new minimum is a merger in the sense of [definition 2.4\(a\)](#). The table writes “merger” when it adjoins values to the head or creates the stack and “new minimum” when $E = \emptyset$, and it writes “head exhausted” for an endpoint choice in a head of size one.

i	π_i	move	m	interval stack	$W_p(L)$
1	9	new minimum, merger	9	$\{10, \dots, 15\}$	$W_8((6))$
2	11	interior choice	9	$\{10\} \mid \{12, \dots, 15\}$	$W_8((1, 4))$
3	10	head exhausted	9	$\{12, \dots, 15\}$	$W_8((4))$
4	14	interior choice	9	$\{12, 13\} \mid \{15\}$	$W_8((2, 1))$
5	5	new minimum, merger	5	$\{6, 7, 8, 12, 13\} \mid \{15\}$	$W_4((5, 1))$
6	12	interior choice	5	$\{6, 7, 8\} \mid \{13\} \mid \{15\}$	$W_4((3, 1, 1))$
7	6	endpoint choice	5	$\{7, 8\} \mid \{13\} \mid \{15\}$	$W_4((2, 1, 1))$
8	2	new minimum, merger	2	$\{3, 4, 7, 8\} \mid \{13\} \mid \{15\}$	$W_1((4, 1, 1))$
9	3	endpoint choice	2	$\{4, 7, 8\} \mid \{13\} \mid \{15\}$	$W_1((3, 1, 1))$
10	8	endpoint choice	2	$\{4, 7\} \mid \{13\} \mid \{15\}$	$W_1((2, 1, 1))$
11	4	endpoint choice	2	$\{7\} \mid \{13\} \mid \{15\}$	$W_1((1, 1, 1))$
12	7	head exhausted	2	$\{13\} \mid \{15\}$	$W_1((1, 1))$
13	1	new minimum	1	$\{13\} \mid \{15\}$	$W_0((1, 1))$
14	13	head exhausted	1	$\{15\}$	$W_0((1))$
15	15	head exhausted	1	$\emptyset$	$W_0(\emptyset)$

The first five rows are [examples 2.3](#) and [2.9](#). In the terms of [lemma 2.11](#), at row 5 the letters 9, 10, and 11 do not separate 8 from 12, because no letter smaller than 8 had been read when they were read, whereas 14, read after $9 < 13$, separates 13 from 15 from row 4 on. The interior choice 12 then creates three intervals, separated between 8 and 13 and between 13 and 15. After the endpoint choice 6, the new minimum 2 adjoins $\{3, 4\}$. The head $\{3, 4, 7, 8\}$ contains the read letters 5 and 6 in its range, again without a separation. At row 8 the unread letters above $m = 2$ are 3, 4, 7, 8, 13, 15, and the lemma cuts this list between 8 and 13, with witness 12 read after $5 < 8$, and between 13 and 15, with witness 14, and nowhere else. This is the stack of the table. The head is then consumed in the 231-avoiding order 3, 8, 4, 7, whose last letter exposes the tail. The new minimum 1 adjoins nothing, and 13, 15 complete the scan. Throughout, the later minima 5, 2, and 1 are read between letters of the stack, and only sizes enter the state: the heads $\{6, 7, 8, 12, 13\}$ of row 5 and $\{3, 4, 7, 8\}$ of row 8 contribute to L only their sizes 5 and 4.

2.3. The exact one-threshold recurrence. Recall that m is the least letter in the scanned prefix, virtual for the empty prefix. Let p be the number of unread letters below m , which we call the *control* of the state, and let $L = (\ell_1, \dots, \ell_s)$ be the positive sizes of the active and deferred intervals. For a legal prefix with these data, let $W_p(L)$ be the number of completions that produce a 1342-avoiding permutation. By [proposition 2.10\(c\)](#), these completions are exactly the sequences

of legal moves that read every unread letter, and by [definition 2.4](#) the legal moves from a legal prefix, their successor states and their multiplicities depend only on (p, L) and on the local ranks of the letters read. Hence this number depends only on (p, L) , as the proof below makes explicit. Write $|L| = \sum_{i=1}^s \ell_i$ and $\text{len}(L) = s$. Let $\text{nz}(x_1, \dots, x_r)$ denote the list obtained by deleting every zero entry. If L' is a list, $(\ell)L'$ and $(a, b)L'$ denote concatenation with the displayed initial entries. For a statement E , $\mathbf{1}_E$ denotes its indicator.

Proposition 2.13 (One-threshold recurrence). *For $L = (\ell)L'$ with $\ell \geq 1$,*

$$\begin{aligned} W_p((\ell)L') &= \sum_{h=0}^{p-1} W_h((\ell + p - 1 - h)L') + E_\ell(p, L') \\ &\quad + \sum_{\substack{a, b \geq 1 \\ a+b=\ell-1}} W_p((a, b)L'), \end{aligned} \quad (5)$$

where

$$E_\ell(p, L') = \begin{cases} W_p(L'), & \ell = 1, \\ 2W_p((\ell - 1)L'), & \ell \geq 2. \end{cases} \quad (6)$$

The boundary and initial conditions are

$$W_p(\emptyset) = \mathbf{1}_{p=0} + \sum_{h=0}^{p-1} W_h(\text{nz}(p - 1 - h)), \quad (7)$$

$$|\text{Av}_n(1342)| = W_n(\emptyset). \quad (8)$$

These equations count every 1342-avoiding permutation exactly once.

Proof. By [proposition 2.10\(c\)](#), the completions that produce a 1342-avoiding permutation are exactly those all of whose moves are legal, and by [definition 2.4](#) with [lemma 2.5\(b\)](#) the legal moves from a legal prefix are the new minima and the letters of the active head. Partition the possible next letters by their location. If the next letter is below m , let h be the number of unread letters below it. There is one letter for each $0 \leq h < p$. The new minimum leaves h unrestricted lower letters, while the $p - 1 - h$ letters between it and m join the active interval. This gives the first line of (5). Among constrained letters, only the active interval can be read legally, by [lemma 2.5\(b\)](#). If the active interval's size is one, it has one endpoint. If its size is at least two, its two endpoints have the same standardized successor. These choices give (6). An interior letter replaces the active interval by ordered subintervals (a, b) with $a, b \geq 1$ and $a + b = \ell - 1$, giving the last line of (5) by [lemma 2.2](#). With an empty stack only a new minimum can be read, which gives (7). Regard each coefficient-1 summand as one transition and the coefficient-2 summand as two labeled transitions, of local ranks 1 and ℓ .

The quantity

$$\rho_1(p, L) = p + |L| \quad (9)$$

decreases by one under every transition. Thus every maximal transition sequence ends at $(0, \emptyset)$. Conversely, the selected local rank at each step recovers a unique next letter in the current ordered alphabet. Maximal transition sequences and avoiding completions are therefore inverse constructions, proving correctness and (8). $\square$

Example 2.14 (All one-threshold moves). After the sixth letter of (4), the unread letters below the current minimum 5 are 1, 2, 3, 4, and the interval stack is $\{6, 7, 8\} \mid \{13\} \mid \{15\}$. The state is $W_4((3, 1, 1))$, and (5) gives

$$\begin{aligned} W_4((3, 1, 1)) &= W_0((6, 1, 1)) + W_1((5, 1, 1)) + W_2((4, 1, 1)) + W_3((3, 1, 1)) \\ &\quad + 2W_4((2, 1, 1)) + W_4((1, 1, 1, 1)). \end{aligned}$$

The first four terms correspond to reading 1, 2, 3, or 4, which adjoin $\{2, 3, 4\}$, $\{3, 4\}$, $\{4\}$, or nothing to the active interval. The fifth term records the two endpoints 6, 8 of the active interval, and the last records its interior letter 7. Neither 13 nor 15 can be read legally until the active interval is empty. The scan of [example 2.12](#) continues with the endpoint 6.

Example 2.15 (Counting $\text{Av}_5(1342)$ by the literal recurrence). By (8) and (7),

$$|\text{Av}_5(1342)| = W_5(\emptyset) = W_0((4)) + W_1((3)) + W_2((2)) + W_3((1)) + W_4(\emptyset),$$

the five terms corresponding to the first letter 1, 2, 3, 4, 5. Expanding by (5) and (6) reaches twenty distinct states in all, listed here with their values, in increasing order of $\rho_1 = p + |L|$:

ρ_1	values
0	$W_0(\emptyset) = 1$
1	$W_0((1)) = 1, W_1(\emptyset) = 1$
2	$W_0((1, 1)) = 1, W_0((2)) = 2, W_1((1)) = 2, W_2(\emptyset) = 2$
3	$W_0((1, 2)) = 2, W_0((2, 1)) = 2, W_0((3)) = 5, W_1((1, 1)) = 3, W_1((2)) = 6, W_2((1)) = 6, W_3(\emptyset) = 6$
4	$W_0((4)) = 14, W_1((3)) = 20, W_2((2)) = 23, W_3((1)) = 23, W_4(\emptyset) = 23$
5	$W_5(\emptyset) = 103$

For instance, $W_0((4)) = 2W_0((3)) + W_0((1, 2)) + W_0((2, 1)) = 10 + 2 + 2 = 14$ (two endpoints and the interior choices $(a, b) = (1, 2), (2, 1)$), $W_1((3)) = W_0((3)) + 2W_1((2)) + W_1((1, 1)) = 5 + 12 + 3 = 20$ (the new minimum adjoins nothing, then endpoints, then the interior choice $(1, 1)$), $W_2((2)) = W_0((3)) + W_1((2)) + 2W_2((1)) = 5 + 6 + 12 = 23$, and $W_3((1)) = W_0((3)) + W_1((2)) + W_2((1)) + W_3(\emptyset) = 5 + 6 + 6 + 6 = 23$, so that $W_5(\emptyset) = 14 + 20 + 23 + 23 + 23 = 103$. The states with $p = 0$ are counted by Catalan numbers, $W_0((\ell)) = C_\ell$, as [lemma 2.2](#) predicts. The number of distinct states grows exponentially with n ([proposition 2.16](#) below). [Example 2.21](#) recomputes the same value from the kernel rows of the shrunk state space.

2.4. The exponential literal state space. The recurrence (5) is finite for each n , but memoizing its composition-valued argument does not give a polynomial bound.

Proposition 2.16. *For $n \geq 2$, the computation of $W_n(\emptyset)$ reaches at least F_{n-1} distinct composition arguments, where F_{n-1} is the $(n-1)$ st Fibonacci number, $F_1 = F_2 = 1$. Consequently, literal memoization of (5) has exponentially many states.*

Proof. Choosing the letter 1 at the initial state reaches $W_0((n-1))$. Repeated interior choices reach every positive composition L satisfying

$$|L| + \text{len}(L) = n. \quad (10)$$

Indeed, reversing an interior split replaces $(a, b, \dots)$ by $(a + b + 1, \dots)$, and repeated reversals return every such composition to $(n-1)$. If L has s parts, then $|L| = n - s$, so the number of choices is $\binom{n-s-1}{s-1}$. Summing over $1 \leq s \leq \lfloor n/2 \rfloor$ gives

$$\sum_{s=1}^{\lfloor n/2 \rfloor} \binom{n-s-1}{s-1} = F_{n-1}. \quad \square$$

The values of the ordered parts cannot be replaced by their total and their number. For example, when $p = 0$, (3) gives $W_0((1, 3)) = C_1 C_3 = 5$ and $W_0((2, 2)) = C_2^2 = 4$, where $C_j = \frac{1}{j+1} \binom{2j}{j}$. The next construction removes the entire composition from the memoization key while retaining its effect on completion counts.

2.5. Protected-tail factorization. Before the general statement, consider the smallest nontrivial instance of (5): one unread letter below the current minimum and a head of size one, with an arbitrary tail L' . Either the head letter is read first, which exposes L' at control 1, or the lower letter is read first and the head letter next, which exposes L' at control 0. Hence

$$W_1((1)L') = W_1(L') + W_0(L').$$

The two coefficients count the ways to reach the exposure of L' at controls 1 and 0, and they do not depend on L' . Writing them as $K_1(1, 1) = K_1(1, 0) = 1$, the identity reads $W_1((1)L') = \sum_t K_1(1, t)W_t(L')$. The theorem below shows systematically when such a factorization holds for a recurrence on a control set and a stack: the transitions before the tail is exposed must leave the tail unchanged and must not depend on it.

Let $\mathcal{C}$ be a set and $\mathcal{B}^*$ the set of finite words over an alphabet $\mathcal{B}$. In the application below, an element of $\mathcal{C}$ records the number of unread values that the obligations so far leave unconstrained, and a word in $\mathcal{B}^*$ lists the sizes of the pending intervals, active head first. A *recurrence* H on $\mathcal{C} \times \mathcal{B}^*$ assigns to each nonboundary state (c, Y) an equation

$$H_c(Y) = \sum_e H_{c_e}(Y_e)$$

with finitely many summands, a summand with coefficient m being written as m equal summands, as in the proof of [proposition 2.13](#). A boundary state has a prescribed nonnegative-integer value and no equation. Each summand is a *transition* $(c, Y) \rightarrow (c_e, Y_e)$, so parallel transitions may occur. A *path* is a finite sequence of consecutive transitions, and a *complete path* is one ending at a boundary state. The recurrence *terminates* if it has no infinite path. Then every state has finitely many complete paths, and induction along the well-founded transition relation shows that $H_c(Y)$ is the sum, over the complete paths from (c, Y) , of the boundary values at their last states.

For a factorization $Y = UL$ of a stack we write $U \mid L$ when the suffix L is the one under consideration. The marker is a notational device and not part of the state. In the application, L is a tail of the interval stack at a separation, chosen freely. A state is an *exposure* of L if its stack is exactly L , that is, if it has the form $(t, \emptyset \mid L)$. Given a state $s_0 = (c, U \mid L)$ with $U \neq \emptyset$, call L *protected before exposure from* s_0 if the following two conditions hold. (P1) Every state reachable from s_0 by a path containing no exposure of L has the form $(c', U' \mid L)$ with $U' \neq \emptyset$. (P2) For every such state and every word $L^* \in \mathcal{B}^*$, replacing L by L^* induces a bijection between the transitions leaving $(c', U' \mid L)$ and those leaving $(c', U' \mid L^*)$, with corresponding targets $(c'', V' \mid L)$ and $(c'', V' \mid L^*)$. Thus (P1) says that the suffix L survives untouched until it is exposed, and (P2) says that the transitions available before exposure do not depend on it. Call L *necessarily exposed from* s_0 if every complete path from s_0 contains an exposure of L , possibly as its last state.

Theorem 2.17 (Protected-tail factorization). *Let H be a terminating recurrence on $\mathcal{C} \times \mathcal{B}^*$. Suppose that, for every $c \in \mathcal{C}$, $x \in \mathcal{B}$, and $L \in \mathcal{B}^*$, the marked suffix L is protected before exposure from $(c, x \mid L)$ and necessarily exposed from that state.*

Let $K_x^L(c, t)$ be the number of paths from $(c, x \mid L)$ that are stopped at their first exposure of L and reach it at $(t, \emptyset \mid L)$. Then $K_x^L(c, t)$ is independent of L . Denote it by $K_x(c, t)$. If $Y = x_1 \cdots x_s$ and $G_t = H_t(\emptyset)$, then

$$H_c(Y) = \sum_t (K_{x_1} K_{x_2} \cdots K_{x_s})(c, t) G_t. \quad (11)$$

Here

$$(AB)(c, t) = \sum_{u \in \mathcal{C}} A(c, u) B(u, t),$$

and the empty product is the identity kernel $\mathbf{1}_{c=t}$. All displayed sums are finite by termination.

Proof. By (P1) and (P2) and induction on the number of transitions, replacing L by L^* gives a bijection between the paths from $(c, x \mid L)$ that contain no exposure of L and the paths from $(c, x \mid L^*)$ that contain no exposure of L^* , with equal controls at corresponding states. The transition that produces the first exposure leaves a state of the form $(c', U' \mid L)$ and is matched as well. Hence $K_x^L(c, t)$ is independent of L . By necessary exposure, every complete path from $(c, x \mid L)$ has a unique first exposure of L , of the form $(t, \emptyset \mid L)$. Cutting the path there gives a stopped path counted by $K_x(c, t)$ followed by a complete path from (t, L) , counted by $H_t(L)$. Concatenation is the inverse operation. Therefore

$$H_c(xL) = \sum_t K_x(c, t) H_t(L).$$

Applying this identity successively to $x_1, \dots, x_s$ proves (11). $\square$

Remark 2.18 (Pushdown systems and pop sequences). Conditions (P1) and (P2) say that the recurrence is a pushdown system: a transition depends only on the control and on the part of the stack before the marker, and the part after it is untouched until exposed, which a pushdown automaton satisfies by definition. The kernel $K_x(c, t)$ counts the runs that pop x while passing from c to t , and [theorem 2.17](#) is the counting form of the classical conversion of a pushdown automaton into a context-free grammar, whose variable $[cx t]$ describes exactly those runs [15, Theorem 6.14], [21, Lemma 2.27]. The same runs are the pop sequences of weighted pushdown systems [20], and (11) is the first-passage decomposition of weighted lattice paths [13, Sections I.5.3 and V.4]. We state and prove the theorem because the control set and the alphabet are infinite in our application, so the textbook statements do not apply verbatim, and because the consequence differs: with infinitely many controls the construction yields an infinite grammar, so no algebraicity follows from it [10], and what the factorization provides is a memoization key of polynomial size ([theorem 1.1](#)) rather than a generating function.

2.6. The scalar kernel algorithm. We apply [theorem 2.17](#) to the recurrence (5), with $\mathcal{C} = \mathbb{N}$ the set of values of p , with $\mathcal{B}$ the set of positive integers, and with W in the role of the recurrence H . A word in $\mathcal{B}^*$ is then the list L of interval sizes. Mark the boundary before an arbitrary old tail L . A new minimum can enlarge the active head, an endpoint can shorten it, and an interior letter can replace it by two new parts. None of these moves changes L before every part preceding the boundary has disappeared. Equations (5) and (6) show that, before exposure, the transitions leaving a state and their targets are determined by p and the blocks preceding the marker. Replacing L therefore gives the bijection of outgoing transitions required by (P2). Every move decreases (9), and the only terminal state has an empty stack. Hence no complete path can terminate before exposing L . Thus [theorem 2.17](#) applies.

For $\ell \geq 1$, let $K_\ell(p, t)$ be the number of paths from $W_p((\ell) \mid L)$ stopped when L is first exposed, at $W_t(\emptyset \mid L)$. The definition is independent of L . Recall that the control p is the number of unread letters below the current minimum. In the kernel we call p the *source control*, t a *terminal control*, and $K_\ell(p, \cdot)$ a *scalar kernel row*. A transition that selects an unread letter below the current minimum is a *base move*. A transition that selects a value from the active head is an *active-interval move*. Then

$$W_p((\ell)L) = \sum_{t \geq 0} K_\ell(p, t) W_t(L). \quad (12)$$

Put $K_0(p, t) = \mathbf{1}_{p=t}$ and $G_p = W_p(\emptyset)$.

Proposition 2.19 (Scalar kernel recurrences). *For $\ell \geq 1$,*

$$\begin{aligned} K_\ell(p, t) &= \sum_{h=0}^{p-1} K_{\ell+p-1-h}(h, t) + D_\ell(p, t) \\ &\quad + \sum_{\substack{a, b \geq 1 \\ a+b=\ell-1}} \sum_{u \geq 0} K_a(p, u) K_b(u, t), \end{aligned} \quad (13)$$

where

$$D_1(p, t) = \mathbf{1}_{p=t}, \quad D_\ell(p, t) = 2K_{\ell-1}(p, t) \quad (\ell \geq 2). \quad (14)$$

The empty-stack values satisfy

$$G_p = \mathbf{1}_{p=0} + \sum_{h=0}^{p-1} \sum_{t \geq 0} K_{p-1-h}(h, t) G_t, \quad |\text{Av}_n(1342)| = G_n. \quad (15)$$

Proof. Partition a stopped path by its first move. A new minimum leaving h lower letters gives the first sum in (13). If $\ell = 1$, consuming the only active letter exposes the old tail at the same control. If $\ell \geq 2$, either endpoint leaves an active interval of size $\ell - 1$. These are the terms in (14). An interior letter creates ordered parts (a, b) with $a + b = \ell - 1$. Re-mark the resulting state as $(p, (a) \mid (b)L)$. A path stopped at the first exposure of L passes through a unique first exposure of $(b)L$, because every transition changes the stack only in its current active head. Cutting there splits the path into one counted by $K_a(p, u)$, for some intermediate control u , and one counted by $K_b(u, t)$. Concatenation inverts the cut. Summing over u gives the last line of (13). Finally, apply (7). A new minimum creates a part of size $p - 1 - h$, to which (12) applies, and K_0 covers the case $h = p - 1$. This proves (15). $\square$

Lemma 2.20 (Exact scalar support). *For every $p \geq 0$ and $\ell \geq 1$,*

$$\text{supp } K_\ell(p, \cdot) = \{0, 1, \dots, p\}. \quad (16)$$

Moreover, $K_\ell(p, p) = C_\ell$.

Proof. A base move strictly decreases the control, while an active-interval move leaves it unchanged. Thus every terminal control is at most p . A path ending at p uses no base move. By lemma 2.2, such paths are in bijection with the 231-avoiding permutations of the original ℓ -element interval, and hence are counted by C_ℓ .

For $t < p$, first choose the unique base letter having exactly t unread base letters below it. The control becomes t and the active interval has size $\ell + p - 1 - t$. Repeatedly selecting an endpoint exhausts that interval without selecting another base letter and exposes the marked suffix at control t . Thus every $t \in \{0, \dots, p\}$ occurs. $\square$

Example 2.21 (From kernel rows to $|\text{Av}_5(1342)|$). Order each row by terminal control. Equations (13)–(14) give

$$K_1(1, \cdot) = (1, 1), \quad K_3(0, \cdot) = (5), \quad K_2(1, \cdot) = (4, 2), \quad K_1(2, \cdot) = (3, 1, 1),$$

for example $K_1(2, \cdot) = K_2(0, \cdot) + K_1(1, \cdot) + \mathbf{e}_2 = (2) + (1, 1) + \mathbf{e}_2 = (3, 1, 1)$, where $\mathbf{e}_t(u) = \mathbf{1}_{u=t}$. The rows of grade 4 follow:

$$\begin{aligned} K_4(0, \cdot) &= 2K_3(0, \cdot) + K_1(0, 0)K_2(0, \cdot) + K_2(0, 0)K_1(0, \cdot) = (10) + (2) + (2) = (14), \\ K_3(1, \cdot) &= K_3(0, \cdot) + 2K_2(1, \cdot) + \sum_u K_1(1, u)K_1(u, \cdot) = (5) + (8, 4) + (2, 1) = (15, 5), \\ K_2(2, \cdot) &= K_3(0, \cdot) + K_2(1, \cdot) + 2K_1(2, \cdot) = (5) + (4, 2) + (6, 2, 2) = (15, 4, 2), \\ K_1(3, \cdot) &= K_3(0, \cdot) + K_2(1, \cdot) + K_1(2, \cdot) + \mathbf{e}_3 = (12, 3, 1, 1). \end{aligned}$$

Since $(G_0, \dots, G_4) = (1, 1, 2, 6, 23)$, (15) gives

$$\begin{aligned} G_5 &= 14G_0 + (15G_0 + 5G_1) + (15G_0 + 4G_1 + 2G_2) + (12G_0 + 3G_1 + G_2 + G_3) + G_4 \\ &= 14 + 20 + 23 + 23 + 23 = 103, \end{aligned}$$

the five groups corresponding, in order, to $h = 0, 1, 2, 3, 4$, the last through the identity kernel K_0 . The five group sums are the five terms $W_0((4)), \dots, W_4(\emptyset)$ of [example 2.15](#): each kernel row compresses the subtree of the literal recurrence below one first-letter choice into a vector indexed by the control at which the empty tail is exposed.

Theorem 2.22 (Polynomial enumeration of 1342-avoiders). *For every $N \geq 0$, the numbers $|\text{Av}_n(1342)|$ for $0 \leq n \leq N$ can be computed using $O(N^5)$ exact-integer arithmetic operations and $O(N^3)$ stored integers. Every integer has $O(N \log N)$ bits. Hence the bit time is $O(N^5 M(N \log N))$ and the bit storage is $O(N^4 \log N)$.*

Proof. Order kernel rows by the grade $p + \ell$. The row in a base term of (13) has grade $h + (\ell + p - 1 - h) = p + \ell - 1$. In a split term, $p + a < p + \ell$. If $K_a(p, u) \neq 0$, then $u \leq p$ by [lemma 2.20](#), and hence $u + b \leq p + b < p + \ell$. The term D_1 is initial data, and D_ℓ for $\ell \geq 2$ uses the row of grade $p + \ell - 1$. Thus increasing grade is a topological order for all kernel dependencies. Equation (15) is then evaluated in increasing p , because every terminal control t in its nonzero terms satisfies $t \leq h < p$.

We compute every row in the *padded region* $p \geq 0, \ell \geq 1, p + \ell \leq N$: it is closed under the dependencies in the preceding paragraph and contains every row with $\ell \geq 1$ needed to evaluate (15) for $n \leq N$, together with rows that no such evaluation reaches. It contains $\binom{N+1}{2} = O(N^2)$ source rows. By [lemma 2.20](#), these rows have at most

$$\sum_{\ell=1}^N \sum_{p=0}^{N-\ell} (p+1) = \binom{N+2}{3} = O(N^3)$$

nonzero entries in total. For one source row, a direct sparse evaluation of all split terms has $O(N)$ choices of a , $O(N)$ possible intermediate controls u , and at most $O(N)$ terminal controls in the row being added. It therefore uses $O(N^3)$ operations per source row and $O(N^5)$ operations in total. The nonsplit kernel terms use $O(N^4)$ operations in total, and (15) uses $O(N^3)$.

For a scalar kernel entry, suffix independence permits us to take the marked suffix to be empty. Its initial number of unread letters is then $p + \ell \leq N$. An empty-stack computation also starts with at most N unread letters. This number decreases at every edge. At each state, the sum of the outgoing transition multiplicities is the number of legal next letters and is at most N . Thus each stored count is at most $(N+1)^N$ and has $O(N \log N)$ bits. Accounting for integer multiplication and for the bit length of every stored entry gives the stated bit bounds. Every product and partial accumulator is a nonnegative contribution to a final kernel or empty-stack value, so the same bound applies to intermediate integers. $\square$

Bóna proved the algebraic generating function [8]

$$\sum_{n \geq 0} |\text{Av}_n(1342)| x^n = \frac{(1 - 8x)^{3/2} - 8x^2 + 20x + 1}{2(1+x)^3}. \quad (17)$$

Thus the scalar ordinary generating function is algebraic. [Theorem 2.22](#) supplies a separate transfer construction that extends to the family: we replace p by d ordered band counts, while the interval stack, its merger rule, and the protected-tail theorem remain unchanged.

3. FROM ONE THRESHOLD TO d THRESHOLDS

After a prefix has been read, let b_j be the smallest final entry of an increasing subsequence of length j contained in that prefix, for $1 \leq j \leq d$. When the prefix contains no such subsequence, use the virtual value $b_j = n + j$. Then

$$b_1 < b_2 < \cdots < b_d.$$

We call $b_1, \dots, b_d$ the *patience-sorting thresholds* of the prefix. Every nonvirtual threshold value has already been read. We call the unread values below b_d the *base values*. They fall into d open bands. Band 0 lies below b_1 , while band i lies between b_i and b_{i+1} for $1 \leq i < d$. Write

$$\mathbf{p} = (p_0, p_1, \dots, p_{d-1})$$

for the band sizes. We again call $\mathbf{p}$ the *control*, write $\|\mathbf{p}\|_1 = p_0 + \cdots + p_{d-1}$ for its *control mass*, and denote the zero control by $\mathbf{0}$. Let $B_0, \dots, B_{d-1}$ be the corresponding unread blocks. Together with the pending intervals, the value order is

$$B_0 < b_1 < B_1 < b_2 < \cdots < b_{d-1} < B_{d-1} < b_d < I_1 < \cdots < I_s. \quad (18)$$

The b_j are read values, not members of the unread blocks. Empty blocks are allowed. As in the one-threshold case (section 2), a base move selects a value from some B_i and an active-interval move selects a value from I_1 . In the terms of definition 2.4, a value of B_i with $i < d - 1$ is a non-trigger move, a value of B_{d-1} is a merger, and a value of I_1 is a split (proposition 4.1(b) below). By lemma 3.1 below, the least d -trigger q of definition 2.4 is b_d whenever b_d is nonvirtual. Initially $\mathbf{p} = (n, 0, \dots, 0)$.

If the next base value x lies in band $i \geq 1$, then x ends an increasing subsequence of length $i + 1$, since $b_i < x$. For $i = 0$, the letter x is an increasing subsequence of length one. In either case, x ends no increasing subsequence of length $i + 2$, since that would imply $b_{i+1} < x$, whereas $x \in B_i$ gives $x < b_{i+1}$. Thus only b_{i+1} changes. A base value creates a new d -trigger exactly when it lies in band $d - 1$. The first $d - 1$ bands update one threshold without creating an obligation. The last band also creates a 231-avoidance obligation.

Lemma 3.1 (Least-trigger threshold). *After a prefix has been read, either b_d is virtual and no d -trigger has been read, or b_d is the smallest value of a d -trigger in the prefix. Consequently, every unread value below b_d belongs to none of the projections onto larger values created by earlier triggers, and hence to no obligation created so far. When b_d is nonvirtual, every unread value above b_d belongs to the projection created by the trigger b_d .*

Proof. By definition, a nonvirtual b_d is the least possible final value of an increasing d -subsequence in the prefix, and hence is itself a d -trigger. Every other d -trigger has value at least b_d . An unread value $x < b_d$ is therefore smaller than every earlier trigger and occurs in none of their projections onto larger values. If b_d is nonvirtual and $x > b_d$ is unread, then x occurs after the trigger b_d and belongs to its projection. If b_d is virtual, no increasing d -subsequence has yet occurred. $\square$

For example, in

$$3\,1\,6\,8\,2\,9\,5\,7\,4,$$

the entries 3, 6 form an increasing 2-subsequence, and the subsequent projection onto values larger than 6 is 8, 9, 7 $\sim$ 231. Hence 3, 6, 8, 9, 7 is an occurrence of $\beta_2 = 12453$. The interspersed values 2, 5, 4, all smaller than 6, do not belong to this projection.

4. THE LITERAL FAMILY RECURRENCE

We now express the band updates in terms of their sizes. For $0 \leq i < d - 1$ and $0 \leq h < p_i$, define

$$T_{i,h}(\mathbf{p}) = (p_0, \dots, p_{i-1}, h, p_{i+1} + p_i - 1 - h, p_{i+2}, \dots, p_{d-1}). \quad (19)$$

The chosen letter becomes the new threshold b_{i+1} . Then h values remain in band i , and the values above it in the old band join band $i + 1$. For the last band put

$$U_h(\mathbf{p}) = (p_0, \dots, p_{d-2}, h), \quad \delta_h = p_{d-1} - 1 - h. \quad (20)$$

For $d = 1$, interpret (20) as $U_h((p_0)) = (h)$. Here the chosen letter is a new least d -trigger. The δ_h unread values above it and below the old b_d join the active stack head.

Let $L = (\ell_1, \dots, \ell_s)$ be the positive pending interval sizes, active head first. For a legal prefix (definition 2.4) with state $(\mathbf{p}, L)$, let $H_{\mathbf{p}}(L)$ be the number of completions that produce a β_d -avoiding permutation. By proposition 4.1(c) below, this number depends only on $(\mathbf{p}, L)$. For an empty stack interpret $\ell_1 = 0$ and its tail as empty.

Proposition 4.1 (State invariant). *Let σ be a legal prefix with thresholds $b_1 < \dots < b_d$, band sizes $\mathbf{p}$, and stack $\mathcal{S}(\sigma) = (I_1, \dots, I_s)$ with sizes $L = (\ell_1, \dots, \ell_s)$.*

- (a) *The ordered layout (18) holds: the unread values below b_d form the d bands, and $I_1 \cup \dots \cup I_s$ is the set of unread values above b_d . A completion w of σ satisfies the residual obligations of all d -triggers of σ exactly when*

$$w|_{I_1 \cup \dots \cup I_s} \in \text{Av}(231)(I_1) \oplus \dots \oplus \text{Av}(231)(I_s); \quad (21)$$

the base values are unrestricted by these obligations.

- (b) *The legal moves from σ , and their effect on $(\mathbf{p}, L)$, are the following. A value of B_i with $i < d - 1$ having h unread values of B_i below it is a non-trigger move to $(T_{i,h}(\mathbf{p}), L)$. A value of B_{d-1} having h unread values of B_{d-1} below it is a merger to $(U_h(\mathbf{p}), \text{nz}(\ell_1 + \delta_h, \ell_2, \dots))$. A value of I_1 of local rank r is a split to $(\mathbf{p}, \text{nz}(r - 1, \ell_1 - r, \ell_2, \dots))$. No other letter is legal.*
- (c) *Consequently, the set of β_d -avoiding completions of σ is determined, up to order isomorphism, by $(\mathbf{p}, L)$.*

Proof. (a) By lemma 3.1, b_d is the least d -trigger of σ when it is nonvirtual, and σ has no d -trigger when b_d is virtual. Hence, by definition 2.4, the stack consists of the unread values above b_d (no value exceeds a virtual $b_d = n + d$, so the two descriptions agree), while the unread values below b_d form the bands by definition. This is (18). The language description (21) is proposition 2.10(a), and the base values lie in no interval.

(b) We classify the unread letters by definition 2.4. Let $x \in B_i$ with $i < d - 1$, and let h be the number of unread values of B_i below x . As observed before lemma 3.1, x ends an increasing subsequence of length $i + 1$ but none of length $i + 2 \leq d$, so x is not a d -trigger of σx : reading it is a non-trigger move, and the stack is unchanged. Replacing b_{i+1} by x leaves h values in band i and moves the other $p_i - 1 - h$ values into band $i + 1$, giving $T_{i,h}(\mathbf{p})$. Next let $x \in B_{d-1}$, with h unread values of B_{d-1} below it. As observed before lemma 3.1 with $i = d - 1$, the letter x ends an increasing d -subsequence (for $d = 1$, x is itself one) and changes only b_d . Moreover, $x < b_d$ makes it smaller than every d -trigger of σ by lemma 3.1. Reading it is therefore a merger, with $q = b_d$ when b_d is nonvirtual. When b_d is virtual, σ has no d -trigger and, by definition 2.4(a), E is the set of all unread values above x , again the unread values strictly between x and $b_d = n + d$. In both cases E has $\delta_h = p_{d-1} - 1 - h$ elements, x becomes the new b_d with h values left in the last band, and the stack becomes $\text{nz}(\ell_1 + \delta_h, \ell_2, \dots)$ by definition 2.4(a). Finally let $x \in I_1$ have local rank r . As noted after definition 2.4, x is a d -trigger of σx exceeding b_d , so reading it is a split. No

threshold changes, since $x > b_d$, and the stack becomes $\text{nz}(r-1, \ell_1-r, \ell_2, \dots)$ by [definition 2.4\(b\)](#). Every other unread letter lies in a deferred interval and is illegal by [lemma 2.5\(b\)](#).

(c) By [proposition 2.10\(c\)](#), the β_d -avoiding completions of σ are exactly the sequences of legal moves that read every unread letter, and by (b) the legal moves from a legal prefix, their successor states and their multiplicities depend only on $(\mathbf{p}, L)$ and on the local ranks of the letters read. Hence, by induction on the number of unread letters, which drops by one at every move, the standardized set of avoiding completions depends only on $(\mathbf{p}, L)$. $\square$

For $(\mathbf{p}, L) \neq (\mathbf{0}, \emptyset)$, partitioning by the next letter gives

$$\begin{aligned} H_{\mathbf{p}}(L) = & \sum_{i=0}^{d-2} \sum_{h=0}^{p_i-1} H_{T_{i,h}(\mathbf{p})}(L) \\ & + \sum_{h=0}^{p_{d-1}-1} H_{U_h(\mathbf{p})}(\text{nz}(\ell_1 + \delta_h, \ell_2, \dots)) \\ & + \min(2, \ell_1) H_{\mathbf{p}}(\text{nz}(\ell_1 - 1, \ell_2, \dots)) \\ & + \sum_{j=2}^{\ell_1-1} H_{\mathbf{p}}((j-1, \ell_1-j, \ell_2, \dots)). \end{aligned} \quad (22)$$

The first double sum is empty when $d = 1$, and the last two lines are absent when the stack is empty. The four lines represent, respectively, an early-band update, a last-band trigger and merger, an endpoint choice in the active interval, and an interior choice. Empty sums vanish. The initial and terminal conditions are

$$a_n^{(d)} := |\text{Av}_n(\beta_d)| = H_{(n,0,\dots,0)}(\emptyset), \quad H_{\mathbf{0}}(\emptyset) = 1. \quad (23)$$

For $d = 1$, identify $H_{(p)}(L) = W_p(L)$. The early-band sum is empty, $U_h((p)) = (h)$, and [\(22\)](#)–[\(23\)](#) reduce to [\(5\)](#)–[\(8\)](#).

Example 4.2 (What the second threshold adds). Specialize to $d = 2$ and scan the permutation π of [\(4\)](#) again. It avoids 12453, since 12453 contains 1342. The table lists, after each letter π_i , the type of move, the thresholds b_1, b_2 , the bands B_0, B_1 , the interval stack, and the state $H_{\mathbf{p}}(L)$, the number of 12453-avoiding completions. A dot marks a virtual threshold, band 0 is the early band and band 1 the last band, an early-band move is a non-trigger move and a last-band move a merger in the sense of [definition 2.4](#), and the labels “merger” and “head exhausted” are as in [example 2.12](#).

i	π_i	move	b_1, b_2	B_0	B_1	interval stack	$H_{\mathbf{p}}(L)$
1	9	early band	9, ·	$\{1, \dots, 8\}$	$\{10, \dots, 15\}$	$\emptyset$	$H_{(8,6)}(\emptyset)$
2	11	last band, merger	9, 11	$\{1, \dots, 8\}$	$\{10\}$	$\{12, \dots, 15\}$	$H_{(8,1)}((4))$
3	10	last band	9, 10	$\{1, \dots, 8\}$	$\emptyset$	$\{12, \dots, 15\}$	$H_{(8,0)}((4))$
4	14	interior choice	9, 10	$\{1, \dots, 8\}$	$\emptyset$	$\{12, 13\} \mid \{15\}$	$H_{(8,0)}((2, 1))$
5	5	early band	5, 10	$\{1, \dots, 4\}$	$\{6, 7, 8\}$	$\{12, 13\} \mid \{15\}$	$H_{(4,3)}((2, 1))$
6	12	endpoint choice	5, 10	$\{1, \dots, 4\}$	$\{6, 7, 8\}$	$\{13\} \mid \{15\}$	$H_{(4,3)}((1, 1))$
7	6	last band, merger	5, 6	$\{1, \dots, 4\}$	$\emptyset$	$\{7, 8, 13\} \mid \{15\}$	$H_{(4,0)}((3, 1))$
8	2	early band	2, 6	$\{1\}$	$\{3, 4\}$	$\{7, 8, 13\} \mid \{15\}$	$H_{(1,2)}((3, 1))$
9	3	last band, merger	2, 3	$\{1\}$	$\emptyset$	$\{4, 7, 8, 13\} \mid \{15\}$	$H_{(1,0)}((4, 1))$
10	8	interior choice	2, 3	$\{1\}$	$\emptyset$	$\{4, 7\} \mid \{13\} \mid \{15\}$	$H_{(1,0)}((2, 1, 1))$
11	4	endpoint choice	2, 3	$\{1\}$	$\emptyset$	$\{7\} \mid \{13\} \mid \{15\}$	$H_{(1,0)}((1, 1, 1))$
12	7	head exhausted	2, 3	$\{1\}$	$\emptyset$	$\{13\} \mid \{15\}$	$H_{(1,0)}((1, 1))$
13	1	early band	1, 3	$\emptyset$	$\emptyset$	$\{13\} \mid \{15\}$	$H_{(0,0)}((1, 1))$
14	13	head exhausted	1, 3	$\emptyset$	$\emptyset$	$\{15\}$	$H_{(0,0)}((1))$
15	15	head exhausted	1, 3	$\emptyset$	$\emptyset$	$\emptyset$	$H_{(0,0)}(\emptyset)$

Compare with the one-threshold scan of [example 2.12](#). The first letter 9 is not a 2-trigger and creates no obligation. It becomes b_1 , and the unread values above it form band 1. The first 2-trigger is 11, whose obligation constrains $\{12, \dots, 15\}$. The value 10 remains a base value, and reading it next makes it the least 2-trigger without adjoining anything. The new minimum 5, a merger for $d = 1$, is here an early-band move: it transfers $\{6, 7, 8\}$ from band 0 to band 1 and leaves the head $\{12, 13\}$ unchanged. It is the 2-trigger 6, not the base value 5, that adjoins $\{7, 8\}$ to the head. The read letters 9 to 12 lie inside the range of the head $\{7, 8, 13\}$, and none of them separates 8 from 13: by [corollary 4.4](#) below, neither 9 nor 11 was read after an earlier 2-trigger smaller than 8, and 10 and 12 were read only after 2-triggers exceeding 8, whereas 14 separates 13 from 15 because it was read after the 2-trigger $10 < 13$. The letter 8, an endpoint choice for $d = 1$, is here an interior choice.

After the eighth letter, the thresholds are $b_1 = 2$ and $b_2 = 6$, the bands are $B_0 = \{1\}$ and $B_1 = \{3, 4\}$, and the stack is $\{7, 8, 13\} \mid \{15\}$. The state is $H_{(1,2)}((3, 1))$. Its next choices are:

next	action	successor
1	update the first threshold	$H_{(0,2)}((3, 1))$
3	new 2-trigger; $\{4\}$ joins the head	$H_{(1,0)}((4, 1))$
4	new 2-trigger; no value is added to the head	$H_{(1,1)}((3, 1))$
7 or 13	endpoint choice in the active interval	$H_{(1,2)}((2, 1))$, twice
8	interior choice with two singleton subintervals	$H_{(1,2)}((1, 1, 1))$
15	illegal until the head is exhausted	illegal

Consequently

$$H_{(1,2)}((3, 1)) = H_{(0,2)}((3, 1)) + H_{(1,0)}((4, 1)) + H_{(1,1)}((3, 1)) + 2H_{(1,2)}((2, 1)) + H_{(1,2)}((1, 1, 1)).$$

The extra threshold is visible in the first move: the choice 1 updates b_1 without creating a 2-trigger and, in this state, transfers no unread value to the second band. The values of an active head need not be read consecutively. Base values may occur between them. The scan continues with 3.

Theorem 4.3 (Literal recurrence). *For every $d \geq 1$, equations (22) and (23) count every β_d -avoiding permutation exactly once.*

Proof. By [proposition 2.10\(c\)](#), the β_d -avoiding permutations are exactly the words all of whose prefixes are legal, and by [proposition 4.1\(b\)](#) the four types of terms in (22) are exactly the legal moves. Local ranks identify letters uniquely, except that two distinct endpoints have the same standardized successor and are recorded with multiplicity two. Interpret this coefficient 2 as two labeled transitions, corresponding to local ranks 1 and ℓ_1 . Moreover

$$\rho(\mathbf{p}, L) = \|\mathbf{p}\|_1 + \sum_{j=1}^s \ell_j \tag{24}$$

drops by one on every move. An early-band move reduces the control mass by one. A last-band move reduces it by $\delta_h + 1$ and adds δ_h to the head. An active-interval move consumes one head value. Hence every maximal recurrence path ends at the unique zero state. Reading the chosen local ranks along such a path reconstructs one permutation, inverse to scanning that permutation. $\square$

The stack can be read off the scanned prefix. Recall that two unread values are adjacent if no unread value lies strictly between them. By [proposition 2.10\(c\)](#), the legal prefixes are exactly

the prefixes of β_d -avoiding permutations, and every legal prefix has an avoiding completion, so condition (iii) below is not vacuous.

Corollary 4.4 (Separations of the stack). *After a legal prefix has been read, let $u < v$ be adjacent unread values above b_d . The following are equivalent: (i) u and v lie in different intervals of the stack, (ii) some letter x with $u < x < v$ was read after a d -trigger smaller than u , (iii) u precedes v in every β_d -avoiding completion. In particular, the stack is determined by the scanned prefix: its union is the set of unread values above b_d by [proposition 4.1\(a\)](#), and its intervals are separated exactly at the adjacent pairs described in (ii). Condition (ii) locates the separations, not the letters that produced them: several letters between u and v may satisfy (ii), and the letter whose split created the separation need not be among them. By [proposition 4.1\(c\)](#), the control $\mathbf{p}$ together with these separations determines the completions up to order isomorphism.*

Proof. Condition (i) implies (iii): by [lemma 2.1](#), an avoiding completion satisfies the residual obligation of every d -trigger of the prefix, hence by [proposition 4.1\(a\)](#) lies in the language (21), which places I_j before I_k for $j < k$. For (ii) implies (iii), let c be the trigger. If v preceded u in a completion, an increasing d -subsequence ending at c , followed by x, v, u , would form β_d , since $c < u < x < v$. For (iii) implies (i), suppose that u and v lie in the same interval, and complete the prefix by the avoiding completion constructed in the proof of [proposition 2.10\(c\)](#): the base values in decreasing order, then the intervals $I_1, I_2, \dots$ in order, each in decreasing order. It avoids β_d and reads v before u .

For (i) implies (ii), we show along the scan that (i) and (ii) agree for every adjacent pair above b_d . The induction repeats that of [lemma 2.11](#) with m replaced by b_d . We give it in full because of the additional early-band case. Initially the stack is empty and no trigger has been read. Let x be the letter read next. In an early-band move, which does not occur when $d = 1$, $x < b_d$ is not a d -trigger and lies below every unread value above b_d , so the stack and both conditions are unchanged. In a last-band move, x becomes the least trigger, and E joins the head or becomes the whole stack when the old stack was empty. An adjacent pair $u < v$ above x that was not an adjacent pair above the old b_d has $u \in E$, so u is smaller than every trigger read before x , and every letter between u and v was read before x . Hence (ii) fails, and (i) fails because u and v lie in the single interval $E \cup I_1$, or E . All other pairs keep their members, their intervals, and the letters between them, and x lies below them, so both conditions are unchanged for them. In an active-interval move the stack is nonempty, so b_d is a d -trigger by [lemma 3.1](#), and $x \in I_1$ is read after it. Only the unread neighbours $u < x < v$ of x , when both exist and $u > b_d$, can form a new adjacent pair. Then u lies in the part of I_1 below x , while v lies in the part above x or, if $x = \max I_1$, is the least element of I_2 . Thus u and v lie in different intervals, and x witnesses (ii) with the trigger $b_d < u$. All other pairs are unaffected. $\square$

Adjacency is needed only in (ii), as the example after [lemma 2.11](#) shows. For arbitrary unread $u < v$ above b_d , conditions (i) and (iii) remain equivalent, and they hold exactly when some pair $a < c$ of adjacent unread values with $u \leq a < c \leq v$ satisfies (ii).

4.1. Why direct memoization is exponential.

Proposition 4.5. *For fixed $d \geq 1$ and $n \geq d + 1$, literal memoization of (22) has at least F_{n-d} reachable composition keys when computing $a_n^{(d)}$, where F_{n-d} is the $(n - d)$ th Fibonacci number, $F_1 = F_2 = 1$. In particular, its state space is exponential in n .*

Proof. From $H_{(n,0,\dots,0)}(\emptyset)$, choose successively the least unread value in bands $0, 1, \dots, d - 1$. Equivalently, begin with $1, 2, \dots, d$. This reaches $H_0((n - d))$. The interior moves are then exactly those used in the proof of [proposition 2.16](#), with n there replaced by $n - d + 1$. They reach every

positive composition L satisfying

$$|L| + \text{len}(L) = n - d + 1,$$

and their number is

$$\sum_{s=1}^{\lfloor (n-d+1)/2 \rfloor} \binom{n-d-s}{s-1} = F_{n-d}. \quad \square$$

5. PROTECTED TAILS AND TRANSFER KERNELS

For the recurrence (22), take $\mathcal{C} = \mathbb{N}^d$ and let the block alphabet be the positive integers. If L is a composition, $(\ell)L$ denotes the result of prepending ℓ . Mark the boundary before L and let $K_\ell^L(\mathbf{p}, \mathbf{t})$ be the number of paths from $H_{\mathbf{p}}((\ell) \mid L)$ stopped when L is first exposed, at $H_{\mathbf{t}}(\emptyset \mid L)$. An early-band move changes only the control. A last-band move can enlarge the active head, an endpoint can shorten it, and an interior letter can replace it by two parts. None of these transitions changes L before exposure, and (22) shows that the transitions leaving a state and their targets depend only on $\mathbf{p}$ and the blocks preceding the marker. Replacing L therefore gives the bijection of outgoing transitions required by (P2). Equation (24) proves termination, and the only terminal state has an empty stack. Hence no complete path can terminate before exposing L . The hypotheses of theorem 2.17 therefore hold.

Corollary 5.1 (Permutation-stack factorization). *The number $K_\ell^L(\mathbf{p}, \mathbf{t})$ is independent of L . Denote it by $K_\ell(\mathbf{p}, \mathbf{t})$. Then*

$$H_{\mathbf{p}}((\ell)L) = \sum_{\mathbf{t}} K_\ell(\mathbf{p}, \mathbf{t}) H_{\mathbf{t}}(L). \quad (25)$$

Consequently, for $L = (\ell_1, \dots, \ell_s)$,

$$H_{\mathbf{p}}(L) = \sum_{\mathbf{t}} (K_{\ell_1} K_{\ell_2} \cdots K_{\ell_s})(\mathbf{p}, \mathbf{t}) H_{\mathbf{t}}(\emptyset). \quad (26)$$

For fixed ℓ and $\mathbf{p}$, the function $K_\ell(\mathbf{p}, \cdot)$ is a *kernel row*. We call $\mathbf{p}$ its *source control*, and an index $\mathbf{t}$ a *terminal control*. Its support is $\text{supp } K_\ell(\mathbf{p}, \cdot) = \{\mathbf{t} : K_\ell(\mathbf{p}, \mathbf{t}) \neq 0\}$. The factorization replaces the composition argument by a product of transfer kernels, evaluated on the required source rows. The next section bounds the number of source and terminal controls required for coefficients through degree N .

Proposition 5.2 (Compressed kernel recurrence). *For $\ell \geq 1$, the transfer kernels satisfy*

$$\begin{aligned} K_\ell(\mathbf{p}, \mathbf{t}) &= \sum_{i=0}^{d-2} \sum_{h=0}^{p_i-1} K_\ell(T_{i,h}(\mathbf{p}), \mathbf{t}) \\ &\quad + \sum_{h=0}^{p_{d-1}-1} K_{\ell+\delta_h}(U_h(\mathbf{p}), \mathbf{t}) + D_\ell(\mathbf{p}, \mathbf{t}) \\ &\quad + \sum_{\substack{a,b \geq 1 \\ a+b=\ell-1}} \sum_{\mathbf{u}} K_a(\mathbf{p}, \mathbf{u}) K_b(\mathbf{u}, \mathbf{t}), \end{aligned} \quad (27)$$

where

$$D_1(\mathbf{p}, \mathbf{t}) = \mathbf{1}_{\mathbf{p}=\mathbf{t}}, \quad D_\ell(\mathbf{p}, \mathbf{t}) = 2K_{\ell-1}(\mathbf{p}, \mathbf{t}) \quad (\ell \geq 2). \quad (28)$$

Proof. Partition a stopped path by its first move. Early- and last-band moves give the first two sums in (27). If $\ell = 1$, consuming the only head value exposes the marked tail at the same control. If $\ell \geq 2$, the two endpoint choices give the same smaller head. These are the terms in (28). An interior choice produces two blocks (a, b) with $a + b = \ell - 1$. Re-mark the resulting state as

$(\mathbf{p}, (a) \mid (b)L)$. As in the scalar case, a path stopped at the first exposure of L passes through a unique first exposure of $(b)L$. Cutting there and summing over the intermediate control gives the kernel product in the last line. $\square$

It remains to connect the kernels to an empty stack. Put

$$G_{\mathbf{p}} = H_{\mathbf{p}}(\emptyset)$$

and define the boundary kernel $K_0(\mathbf{p}, \mathbf{t}) = \mathbf{1}_{\mathbf{p}=\mathbf{t}}$.

Corollary 5.3 (Empty-stack recurrence). *The empty-stack values satisfy*

$$\begin{aligned} G_{\mathbf{p}} = \mathbf{1}_{\mathbf{p}=\mathbf{0}} &+ \sum_{i=0}^{d-2} \sum_{h=0}^{p_i-1} G_{T_{i,h}(\mathbf{p})} \\ &+ \sum_{h=0}^{p_{d-1}-1} \sum_{\mathbf{t}} K_{\delta_h}(U_h(\mathbf{p}), \mathbf{t}) G_{\mathbf{t}}. \end{aligned} \quad (29)$$

Proof. Apply the empty-stack part of (22). When a last-band move creates a head of size δ_h , use (25). The identity kernel K_0 handles $\delta_h = 0$. $\square$

The desired answer is

$$a_n^{(d)} = G_{(n,0,\dots,0)}. \quad (30)$$

6. THE FAMILY ALGORITHM AND ITS COMPLEXITY

Again give a kernel source $(\mathbf{p}, \ell)$ the grade

$$w = \|\mathbf{p}\|_1 + \ell.$$

We verify below that increasing grade is a topological order for the kernel dependencies in (27), and that increasing control mass orders the empty-stack recurrence (29). The band terms of (27) lower the grade by one. For example,

$$\|U_h(\mathbf{p})\|_1 + (\ell + \delta_h) = \|\mathbf{p}\|_1 - (\delta_h + 1) + \ell + \delta_h = w - 1.$$

The split term is controlled by the following support bound.

Lemma 6.1 (Kernel support). *For every $\ell \geq 1$,*

$$\text{supp } K_{\ell}(\mathbf{p}, \cdot) \subseteq \{\mathbf{p}\} \cup \{\mathbf{t} : \|\mathbf{t}\|_1 \leq \|\mathbf{p}\|_1 - 1\}. \quad (31)$$

Proof. Every base move consumes a letter and hence strictly lowers control mass. Active-interval moves leave the control unchanged. If no base move occurs, the terminal control is $\mathbf{p}$. If at least one base move occurs, the terminal control has mass at most $\|\mathbf{p}\|_1 - 1$. $\square$

The dynamic program uses the control coordinates above. Each row operation below is coefficientwise, and every uninitialized row is zero.

Kernel phase. For $w = 1, 2, \dots, N$, and every $\mathbf{p} \in \mathbb{N}^d$, $\ell \geq 1$ with $\|\mathbf{p}\|_1 + \ell = w$, evaluate (27) coefficientwise and store the resulting sparse row. In each split term, for every $a, b \geq 1$ with $a + b = \ell - 1$ and $\mathbf{u} \in \text{supp } K_a(\mathbf{p}, \cdot)$, add the scalar multiple $K_a(\mathbf{p}, \mathbf{u})K_b(\mathbf{u}, \cdot)$ to the row being computed.

Empty-stack phase. Set $G_0 = 1$. For $m = 1, 2, \dots, N$, and every $\mathbf{p} \in \mathbb{N}^d$ with $\|\mathbf{p}\|_1 = m$, evaluate (29), using K_0 as the identity kernel. Return $(G_{(n,0,\dots,0)})_{0 \leq n \leq N}$.

In the kernel phase, the early- and last-band rows in (27) have grade $w - 1$. The term D_1 is initial data, while D_ℓ for $\ell \geq 2$ uses a row of grade $w - 1$. In a split term, $\|\mathbf{p}\|_1 + a < w$ and, by lemma 6.1, $\|\mathbf{u}\|_1 + b \leq \|\mathbf{p}\|_1 + b < w$. Hence increasing w is a topological order. In the empty-stack phase every terminal control used at mass m has smaller mass, including $U_h(\mathbf{p})$ when K_0 occurs. Thus increasing m is topological.

For coefficients through size N , we use the padded region

$$\mathbf{p} \in \mathbb{N}^d, \quad \ell \geq 1, \quad \|\mathbf{p}\|_1 + \ell \leq N. \quad (32)$$

The region contains exactly

$$\binom{N+d}{d+1} = O_d(N^{d+1}) \quad (33)$$

rows, by stars and bars. By lemma 6.1, a row has at most $1 + \binom{\|\mathbf{p}\|_1 + d - 1}{d} = O_d(N^d)$ possible terminal controls.

Proof of theorem 1.1. Correctness follows from theorem 4.3 and corollary 5.1: equations (27) and (29) are the first-step evaluations of the exact factorization (25), and the preceding topological orders evaluate all of their dependencies. It remains to bound the resources.

There are $O_d(N^{d+1})$ source rows and $O_d(N^d)$ possible entries per row, giving $O_d(N^{2d+1})$ stored integers. In a split convolution there are $O(N)$ choices of a , $O_d(N^d)$ intermediate controls $\mathbf{u}$, and $O_d(N^d)$ entries in the row added for each $\mathbf{u}$. Thus a direct sparse implementation uses $O_d(N^{2d+1})$ operations per source row and $O_d(N^{3d+2})$ over all kernels. The nonsplit part uses $O_d(N)$ row additions per source and therefore $O_d(N^{2d+2})$ operations in total. For each of the $O_d(N^d)$ empty-stack controls, (29) sums at most $O(N)$ choices over $O_d(N^d)$ terminal controls, so the G table costs $O_d(N^{2d+1})$. For $d \geq 1$, the exponents $2d + 2$ and $2d + 1$ are both smaller than $3d + 2$.

For a complete path in (22), define its workload by (24). For a stopped kernel path, define its workload to be the control mass plus the sum of the sizes of the blocks preceding the marked suffix. Each initial workload is at most N and decreases by one at every transition. At a state of (22), the number of outgoing transitions is the number of legal next letters and is at most the workload. Hence a path has at most N edges and at most N labeled choices at each nonterminal state. Each stored $G_{\mathbf{p}}$ counts complete paths in (22), while each stored kernel entry counts stopped kernel paths. Every such integer is at most $(N + 1)^N$ and has $O(N \log N)$ bits. Every intermediate product and partial sum is a nonnegative contribution to a stored value and obeys the same bound. Replacing integer multiplication by its $M(N \log N)$ bit cost proves the stated bit time. The bit storage is $O_d(N^{2d+2} \log N)$. $\square$

7. THE TWO-THRESHOLD QUOTIENT FOR 12453

For $d = 2$, write $\mathbf{p} = (p, q)$. Two exact symmetries of the kernels are available in this case, and they serve different purposes. The first is a translation in the lower threshold coordinate: a kernel entry depends on the first coordinates of its source and terminal controls only through their difference, so one coordinate can be dropped from the table. This is the symmetry that improves the direct $d = 2$ specialization $O(N^8)$ and $O(N^5)$ of theorem 1.1 to the $O(N^7)$ and $O(N^4)$ of corollary 7.4. The second is a partial translation in the upper coordinate. It leaves those exponents unchanged and halves the leading constant in the storage count. We first determine the exact support of a kernel row: the bound of lemma 6.1 has the following exact refinement.

Lemma 7.1 (Exact two-threshold support). *For every $\ell \geq 1$,*

$$\text{supp } K_\ell((p, q), \cdot) = \{(p, q)\} \cup \{(u, v) : 0 \leq u \leq p, v \geq 0, u + v \leq p + q - 1\}. \quad (34)$$

Proof. A base move never increases the first coordinate and strictly lowers the coordinate sum, proving necessity. The terminal control (p, q) is reached by using no base move and selecting active-interval endpoints until the interval is empty. To reach a control (u, v) in the second set with $u < p$, first update the minimum so that u unread values remain below it. This gives (u, Q) with $Q = p + q - 1 - u$. The support inequality says $v \leq Q$. If $v = Q$, make no further base move. If $v < Q$, make the last-band move with local parameter $h = v$. If $u = p$, then $v \leq q - 1$, so make that last-band move directly. Finally, exhaust each active interval by repeated endpoint choices. This reaches the marked suffix at every stated terminal control. $\square$

We now prove the first translation.

Lemma 7.2 (First-coordinate translation). *For $\ell \geq 1$ and $0 \leq c \leq p$,*

$$K_\ell((p, q), (c, s)) = K_\ell((p - c, q), (0, s)). \quad (35)$$

Proof. Along a kernel path ending at (c, s) , the first control coordinate never falls below c . Delete the c lowest unread values throughout the path and shift all remaining labels down by c . This preserves the active interval sizes, every transition multiplicity, and the second control coordinate, and gives a path from $(p - c, q)$ to $(0, s)$. The inverse operation adjoins c unread values below every value used by the path. These added values are never selected, because the terminal first coordinate is c . They shift by exactly c the parameter h of every band-0 letter, leave the parameter r of every last-band letter and the local ranks within the active head unchanged, and preserve every transition multiplicity. The two operations are inverse. $\square$

Write

$$R_{\ell,a}(q, s) := K_\ell((a, q), (0, s)). \quad (36)$$

We call $R_{\ell,a}(q, s)$ a *reduced entry* and the table of all such entries the *first-coordinate quotient*. From here on the letter a denotes a reduced first control coordinate, not a block size as in (5) and (27). Block sizes in a split are written ℓ_1, ℓ_2 . By lemma 7.2, every kernel entry is represented by one such reduced entry, with $a = p - c$. For a reduced source (ℓ, a, q) with $a = 0$, the possible terminal coordinates are $0 \leq s \leq q$. If $a > 0$, a base move is necessary and $0 \leq s \leq a + q - 1$. The first-coordinate quotient therefore has

$$\binom{N+2}{3} \text{ rows, } \frac{N(N+1)(N^2+N+4)}{12} \text{ entries.} \quad (37)$$

The entry formula follows by summing $q + 1$ for $a = 0$ and $a + q$ for $a > 0$ over the same padded source region. In particular, it already gives $O(N^4)$ storage. The operation count will be recorded after the second translation.

We first rewrite the kernel recurrence in the reduced variables. In the notation of (36), apply lemma 7.2 to (27). An early-band move from (a, q) gives $R_{\ell,h}(a + q - h - 1, s)$, and a last-band move gives $R_{\ell+q-r-1,a}(r, s)$. In a split with intermediate control (c, m) , the first factor $K_{\ell_1}((a, q), (c, m))$ vanishes unless $c \leq a$, since both parts of the support (34) have first coordinate at most a . For $0 \leq c \leq a$ the two factors become

$$R_{\ell_1,a-c}(q, m) R_{\ell_2,c}(m, s).$$

Writing $a_1 = a - c$ and $a_2 = c$ therefore gives the reduced recurrence

$$\begin{aligned} R_{\ell,a}(q, s) = & \sum_{h=0}^{a-1} R_{\ell,h}(a+q-h-1, s) + \sum_{r=0}^{q-1} R_{\ell+q-r-1,a}(r, s) + D_{\ell,a}(q, s) \\ & + \sum_{\substack{\ell_1, \ell_2 \geq 1 \\ \ell_1 + \ell_2 = \ell - 1}} \sum_{\substack{a_1, a_2 \geq 0 \\ a_1 + a_2 = a}} \sum_{m \geq 0} R_{\ell_1, a_1}(q, m) R_{\ell_2, a_2}(m, s), \end{aligned} \quad (38)$$

where

$$D_{1,a}(q, s) = \mathbf{1}_{a=0, q=s}, \quad D_{\ell,a}(q, s) = 2R_{\ell-1,a}(q, s) \quad (\ell \geq 2),$$

and entries outside the support above are zero.

The reduced grade $\ell + a + q$ is a topological order for (38): every term of the first two sums and every D term has reduced grade $\ell + a + q - 1$, while in a split the first factor has grade $\ell_1 + a_1 + q \leq \ell + a + q - 2$ and, since a nonzero first factor forces $m \leq q$ when $a_1 = 0$ and $m \leq a_1 + q - 1$ when $a_1 > 0$, the second factor has grade $\ell_2 + a_2 + m \leq \ell + a + q - 2$.

The second control coordinate admits a partial quotient as well: raising both the second source coordinate and the terminal coordinate by one leaves a reduced entry unchanged whenever the terminal coordinate is at least $a - 1$.

Lemma 7.3 (Second-coordinate translation). *For $\ell \geq 1$ and $a, q, s \geq 0$, whenever $s \geq a - 1$,*

$$R_{\ell,a}(q+1, s+1) = R_{\ell,a}(q, s). \quad (39)$$

Proof. Put $\gamma = \ell + a + q + 1$, the reduced grade of the left-hand side, and induct on $\gamma \geq 2$. If $\gamma = 2$, then $(\ell, a, q) = (1, 0, 0)$, and (38) gives

$$R_{1,0}(1, s+1) = \mathbf{1}_{s=0} = R_{1,0}(0, s).$$

Assume henceforth that $\gamma > 2$ and that the claim holds at every smaller reduced grade. The h th term of the first sum on the left is $R_{\ell,h}(a+q-h, s+1)$. Since $s \geq a - 1 \geq h - 1$, induction translates this to $R_{\ell,h}(a+q-h-1, s)$, the corresponding term on the right.

In the second sum, the $r = 0$ term on the left vanishes: at second source coordinate zero the largest possible terminal coordinate is 0 if $a = 0$, and $a - 1$ otherwise, whereas the terminal coordinate satisfies $s+1 \geq 1$ and $s+1 \geq a$. Put $r = j+1$ in the remaining terms. Induction gives

$$R_{\ell+q-j-1,a}(j+1, s+1) = R_{\ell+q-j-1,a}(j, s),$$

which gives exactly the terms $0 \leq j < q$ on the right. The D terms also match: for $\ell = 1$ both are $\mathbf{1}_{a=0, q=s}$, while for $\ell > 1$ the claim follows from induction applied to $2R_{\ell-1,a}$.

It remains to compare one split with $a = a_1 + a_2$. On the left it contributes

$$\sum_{m \geq 0} R_{\ell_1, a_1}(q+1, m) R_{\ell_2, a_2}(m, s+1).$$

The $m = 0$ term vanishes by the support of the second factor. Put $m = k+1$. If that second factor is nonzero, then $k \geq a_1 - 1$. Indeed, when $a_2 = 0$, support gives $k \geq s \geq a_1 - 1$, while for $a_2 > 0$ it gives $k \geq s+1-a_2 \geq a_1$. Also $s \geq a - 1 \geq a_2 - 1$. Thus the induction hypothesis applies to both factors.

Both induction calls are at smaller reduced grade. The first has grade $\ell_1 + a_1 + q + 1 < \gamma$. Nonvanishing of that factor also gives

$$k \leq q \quad \text{if } a_1 = 0, \quad k \leq a_1 + q - 1 \quad \text{if } a_1 > 0,$$

by (34). Hence the second factor has reduced grade $\ell_2 + a_2 + k + 1 < \gamma$. Induction therefore gives

$$R_{\ell_1, a_1}(q+1, k+1) R_{\ell_2, a_2}(k+1, s+1) = R_{\ell_1, a_1}(q, k) R_{\ell_2, a_2}(k, s).$$

The same upper and lower support bounds hold for every nonzero term on the right, so this is a bijective reindexing of the split sum. The induction is complete. $\square$

The hypothesis $s \geq a - 1$ cannot be weakened: equation (38) gives $R_{1,2}(1, 1) = 3$ but $R_{1,2}(0, 0) = 2$.

For $a = 0$ and $0 \leq s \leq q$, lemma 7.3 gives $R_{\ell,0}(q, s) = R_{\ell,0}(q - s, 0)$. For $a \geq 1$ and $s \geq a$, it gives

$$R_{\ell,a}(q, s) = R_{\ell,a}(q - s + a - 1, a - 1),$$

where the support condition ensures that the new second source coordinate is nonnegative. Thus we retain only terminal coordinate $s = 0$ when $a = 0$, and terminal coordinates $0 \leq s < a$ when $a \geq 1$. The exact number of retained entries is

$$\binom{N+1}{2} + \binom{N+2}{4} = \frac{N(N+1)(N^2+N+10)}{24}. \quad (40)$$

Thus the second translation halves the leading storage constant relative to the first-coordinate quotient. Exact polynomials for loop iterations and stored entries are recorded in the computation report in the supplement.

Corollary 7.4 (Sharpened 12453 bound). *The numbers $|\text{Av}_n(12453)|$ for $0 \leq n \leq N$ can be computed using $O(N^7)$ exact-integer arithmetic operations and $O(N^4)$ stored integers. The bit time is $O(N^7 M(N \log N))$, and the bit storage is $O(N^5 \log N)$.*

Proof. Use the first-coordinate quotient in this operation count, evaluating (38) in increasing reduced grade, which the preceding paragraphs showed to be a topological order. It has $O(N^3)$ source rows and $O(N^4)$ stored entries by (37). A direct split evaluation has $O(N)$ choices for the block split ℓ_1 , $O(N)$ choices for the control split a_1 , and $O(N)$ possible intermediate second coordinates m . For each such triple, adding the row $R_{\ell_2, a_2}(m, \cdot)$ uses $O(N)$ possible terminal coordinates. Thus one source row costs $O(N^4)$ operations and all split terms cost $O(N^7)$. The second-coordinate translation can reduce the storage constant. Each omitted entry is recovered from (39) in $O(1)$ operations. Direct evaluation of the nonsplit kernel terms costs at most $O(N^5)$. The empty-stack table has $O(N^2)$ source controls, $O(N)$ last-band choices, and $O(N^2)$ possible terminal controls, so it also costs $O(N^5)$. The path-size argument in the proof of theorem 1.1 gives $O(N \log N)$ bits per integer, yielding the asserted bit bounds. $\square$

8. CERTIFIED COMPUTATION FOR $\text{Av}(12453)$

The reduced recurrence was evaluated modulo pairwise distinct primes and then reconstructed over the integers. The programs `av12453_fast_rns.cpp` and `reconstruct_av12453_rns.py` in the supplement perform the modular recurrence evaluation, verification of the coefficient bound, and reconstruction. Separately written implementations provide error-detection checks of the multi-prime program. They are listed at the end of this section. Complete source, execution logs, residues, and machine-readable output are included in the supplement.

Proposition 8.1 (Computer-assisted certification). *The file `code/data/av12453_terms_0_150.txt` in the supplement contains the exact values $|\text{Av}_n(12453)|$ for every $0 \leq n \leq 150$.*

Proof. Put $b_m = |\text{Av}_m(1342)|$ and

$$B_N := \sum_{m=0}^N \binom{N}{m}^2 b_m. \quad (41)$$

Deleting the left-to-right minima of a 12453-avoider leaves a 1342-avoider [9]: a 1342 among the remaining entries could be preceded by an earlier smaller entry, producing a 12453. If m

entries remain, choose their positions, their values, and their standardization. The deleted minima occupy the remaining positions, and their remaining values are forced in decreasing order. Hence $|\text{Av}_N(12453)| \leq B_N$.

The integers b_m are generated exactly from (17). The source `av12453_fast_rns.cpp` implements (38) and the empty-stack recurrence (29) modulo each selected prime. By [theorem 4.3](#), [corollaries 5.1 and 5.3](#), [proposition 5.2](#), and [lemmas 7.2 and 7.3](#), its outputs are the residues of the desired coefficients. The moduli are the twenty-four largest primes below 2^{31} , recorded with the residue packs in the supplement. The program `reconstruct_av12453_rns.py` verifies that the product $\mathcal{M}$ of the eighteen largest exceeds B_{150} . In fact $\mathcal{M}/B_{150} > 1.92$. Since the nonnegative bound B_N is nondecreasing, every coefficient through degree 150 lies in $[0, \mathcal{M})$. The Chinese remainder theorem therefore determines each coefficient uniquely.

The bound and the Chinese remainder theorem therefore give the unique integer lifts. The six remaining primes, withheld from reconstruction, agree with every reconstructed coefficient. The separately written single-modulus $d = 2$ program computes all 151 coefficients through degree 150 modulo $2^{61} - 1$ and also agrees. These are additional error-detection checks, not inputs to the uniqueness argument. $\square$

Brute-force enumeration and (22) give the same counts through $n = 8$ for $d = 1, 2, 3, 4$. Separately written Python and C++ implementations of the unreduced family recurrence agree with one another through $n = 10$. For $d = 2$, the coefficients through $n = 38$ agree with OEIS A116485 and Biers-Ariel's composition recurrence [6, 7, 18]. The first-coordinate quotient and the unreduced family kernel give identical $d = 2$ coefficients through $n = 40$. The dense modular program, which uses neither the second translation nor the storage quotient from [lemma 7.3](#), and the single-modulus program give identical residues modulo $2^{61} - 1$ through $n = 60$.

8.1. Uniform random generation. The transfer tables also generate uniformly random avoiders. By [theorem 4.3](#), the labeled complete paths of (22) from the initial state $H_{(n,0,\dots,0)}(\emptyset)$, with the two endpoint choices counted as distinct transitions as in its proof, correspond bijectively to the β_d -avoiding permutations of length n . By the recursive method [17], such a path is uniformly random when every transition is chosen with probability proportional to the number of complete paths through it, that is, to the value of the corresponding summand of (22). That value is a completion count $H_{\mathbf{p}}(L)$ for a stack L of arbitrary length, which the factorization (26) expresses through the stored kernels. Equivalently, one samples a complete path of the factorized recurrence itself. The first-move partitions in the proofs of [proposition 5.2](#) and [corollary 5.3](#) identify the summands of (27) and (29), with the coefficient-2 endpoint term counted as two summands, with a partition of the stopped and complete paths, and iterating the cut-and-concatenate bijection of [theorem 2.17](#) maps these paths onto the complete paths of (22). At an empty-stack state, a summand of (29) is chosen with probability proportional to its value. This selects a base letter and, for a last-band move, the terminal control $\mathbf{t}$ at which the new head will have been exhausted. Inside a kernel $K_{\ell}(\mathbf{p}, \mathbf{t})$, a summand of (27) is chosen in the same way. This selects a base move, an endpoint of the active head, or an interior letter of it together with the intermediate control $\mathbf{u}$ of the split, and each pending block carries the control at which it must be exposed. By induction on the grade, a given stopped path of $K_{\ell}(\mathbf{p}, \mathbf{t})$ is produced with probability $1/K_{\ell}(\mathbf{p}, \mathbf{t})$ and a given complete path from $G_{\mathbf{p}}$ with probability $1/G_{\mathbf{p}}$. At $\mathbf{p} = (n, 0, \dots, 0)$ this is $1/|\text{Av}_n(\beta_d)|$.

Proposition 8.2 (Uniform random generation). *Let $N \geq n \geq 0$. Once the first-coordinate quotient $R_{\ell,a}(q, s)$ of [section 7](#) and the empty-stack values $G_{(p,q)}$ have been computed for size N , a uniformly random element of $\text{Av}_n(12453)$ can be produced with $O(n^4)$ arithmetic operations on the stored integers.*

Proof. Uniformity is the preceding paragraph. A permutation of length n is produced by n letter choices. Each choice scans the summands of one instance of (29) or (27) in a fixed order, the endpoint term counted as two summands, and stops at the first summand whose running total exceeds a uniformly random integer U with $0 \leq U < T$, where T is the stored value of the state. Every summand is one stored entry, twice one, or a product of two, by lemma 7.2. Every state reached from $(n, 0)$ has grade at most n . At an empty-stack state the last-band terms range over the band position and the terminal control, $O(n^3)$ summands. Inside a kernel the interior terms range over the split position and the intermediate control, again $O(n^3)$. The other groups are smaller. Hence each choice costs $O(n^3)$ operations and a sample costs $O(n^4)$. $\square$

Unlike the coefficients of proposition 8.1, our sampler runs in floating point. Proposition 8.2 is the exact-arithmetic statement. The implementation evaluates the two tables in binary64 with an exact power-of-two scaling per grade, so that the entries up to $N = 300$ stay within range. Every summand of the reduced recurrence is nonnegative, so no cancellation occurs, and the floating-point values $G_{(n,0)}$ agree with the certified coefficients of proposition 8.1 through $n = 150$ to about 10^{-11} relative error. These comparisons are an empirical accuracy check. We have not established a rigorous bound on the distance of the sampling distribution from uniformity. Every sample is verified to avoid 12453 by an independent test based on lemma 2.1, and the sampler was checked against exhaustive enumeration for small n . Building the tables for $N = 300$ took about four hours on eight cores. One million samples of length 300 then took about thirty-five minutes. Figure 2 shows the resulting position–value heatmap, rendered with the script that draws the heatmaps of the PermPAL database [5, 1], beside one of the samples. Its structure reflects $\beta_2 = 1 \oplus \beta_1$: the subword of entries larger than the first letter avoids 1342, whose growth rate 8 [8] is smaller than the growth rate $9 + 4\sqrt{2}$ of $\text{Av}(12453)$ (section 10), which is consistent with the first letter being typically close to n . Its mean over the sample is 291.9. The dark band from the top left to the bottom right shows that a typical avoider stays close to a decreasing curve. The left-to-right minima, whose deletion leaves a 1342-avoider [9], lie below it, and their mean number is 77.5. The single sample shows the same features: a decreasing band of letters with the left-to-right minima scattered below it, and above it the letters exceeding the first letter, which form a 1342-avoider.

9. FORMAL VERIFICATION

The counting recurrences of this paper for $d \leq 2$ have been verified in the Lean 4 proof assistant [12] with the Mathlib library [22]. The development, in the directory `formal/` of the supplement, works with words over $\mathbb{N}$ and 0-based values. It defines pattern containment as the existence of an order-isomorphic subsequence, a finite set of 0-based words in bijection with $\text{Av}_n(\tau)$, the one-threshold recurrence W of (5), the literal recurrence H of (22) for $d = 2$, and the kernel recurrences (13)–(15) for $d = 1$ and (27)–(29) for $d = 2$. In the two kernel systems every sum over controls is restricted to the controls of mass at most $\|\mathbf{p}\|_1$ (to $\{0, \dots, p\}$ for $d = 1$), which lemmas 2.20 and 6.1 show loses nothing. Its four main theorems are, for every n ,

$$\begin{aligned} W_n(\emptyset) &= |\text{Av}_n(1342)|, & G_n &= |\text{Av}_n(1342)|, \\ H_{(n,0)}(\emptyset) &= |\text{Av}_n(12453)|, & G_{(n,0)} &= |\text{Av}_n(12453)|, \end{aligned}$$

that is, theorem 4.3 for $d \leq 2$ and the correctness of the recurrences evaluated by the algorithm of section 6 ((15) and (30)) for 1342 and 12453. Their Lean names are `av1342_count`, `av1342_count_kernel`, `av12453_count`, and `av12453_count_kernel`.

The Lean project `formal/Av12453/` consists of twenty modules in three groups. The modules `Basic.lean`, `Trigger.lean`, and `FirstLetter.lean` define words, order isomorphism, containment, the avoiders, and the patterns β_d , and prove lemmas 2.1 and 2.2. The eight modules of `OneT`

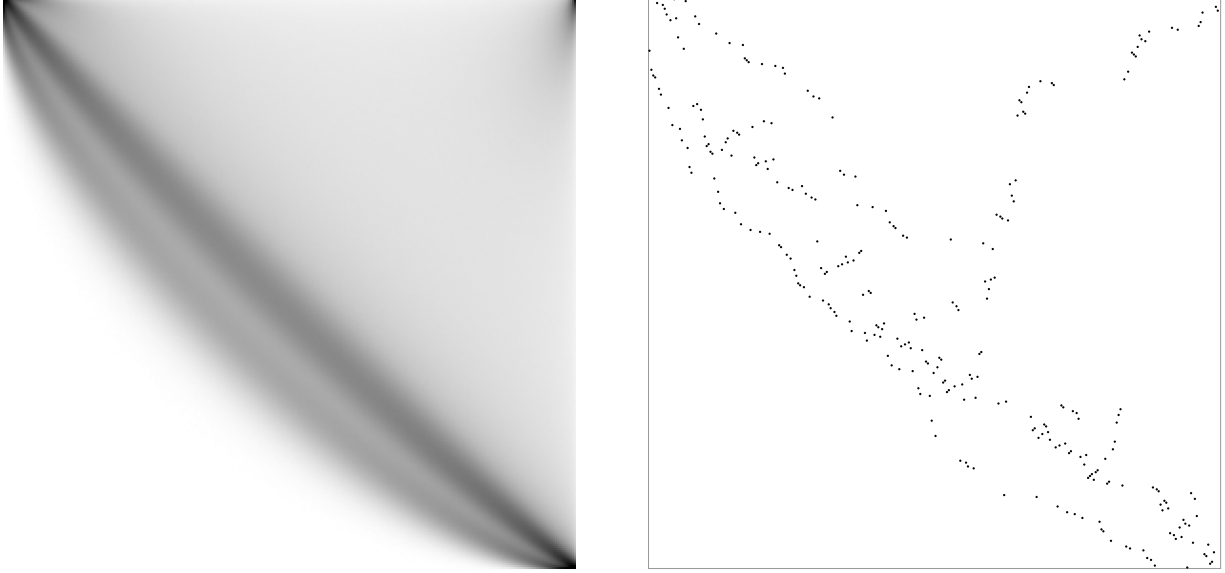


FIGURE 2. Left: position–value heatmap of one million elements of $\text{Av}_{300}(12453)$ drawn by the floating-point implementation of the uniform sampler of [section 8.1](#). The cell at position i (left to right) and value v (bottom to top) is shaded by the square root of the number of samples with $\pi_i = v$, from white (no sample) to black (the largest cell count, 70 204). Right: one of the samples, with a dot at (i, π_i) in the same orientation. Its first letter is 272 and it has 71 left-to-right minima.

hreshold/ treat $d = 1$. The module `Defs.lean` defines the scan, its state (p, L) , and W . The modules `Invariant.lean`, `Semantics.lean`, and `Counting.lean` prove the separation invariant of [lemma 2.11](#), parts (b) and (c) of [proposition 2.10](#), and `av1342_count`. The modules `Kernel.lean`, `KernelSupport.lean`, `KernelFactor.lean`, and `KernelCount.lean` define K , D , and G , and prove [lemma 2.20](#), the factorization (12), and `av1342_count_kernel`. The nine modules of `TwoThreshold/` repeat this for $d = 2$, with `Thresholds.lean` for the two thresholds and their bands ([section 3](#)) and `Defs.lean` for H . The directory also contains `Axioms.lean`, which walks every constant declared in the twenty modules and fails if any depends on `sorryAx` or on an axiom other than propositional extensionality, choice, and quotient soundness. The script `check.sh` builds the project, requires the build to be free of warnings, runs the axiom sweep, and rejects any occurrence of `sorry` or `native_decide` in the sources. There is also a `README.md` with a table of the modules and their headline theorems, and an independent audit report. The Lean toolchain (version 4.33.1) and the Mathlib revision are pinned in the project files.

Two things are left to a human reader. The first is to run `check.sh`, or equivalently `lake build` followed by the axiom sweep, and see it end with the line `CERTIFICATE CHECK PASSED`. The second is to read, against this paper, the definitions that occur in the four theorem statements. Nothing else needs to be read to know what the theorems say. They are listed by file. The module `Basic.lean` defines `OrderIso`, `Contains`, `Avoids`, `IsPermOf`, `iota`, and `beta`, so that `beta 2` is the 0-based word 0, 1, 3, 4, 2 of 12453. The module `OneThreshold/Defs.lean` defines `perms`, `avoiders`, `nz`, and W , and `TwoThreshold/Defs.lean` defines H . The module `OneThreshold/Kernel.lean` defines K , D , and G for $d = 1$, and `TwoThreshold/Kernel.lean` defines the control sets `ctrls` and S and K , D , and G for $d = 2$. These definitions occupy about seventy lines of Lean in all. The recurrences are written with an explicit recursion bound: W , H , K , and G evaluate an auxiliary function at a bound exceeding the grade of the argument, and

the development proves that every sufficient bound gives the same value, so the reader need only compare the equations of the auxiliary functions with the displays (5), (22), (13)–(15), and (27)–(29). Each definition carries a docstring giving this dictionary, with the 0-based conventions spelled out, and the displays are also restated as proved theorems (`K_succ_eq`, `G_eq`, and their analogues), which serve as reading aids but are not part of what must be trusted.

The proofs follow the paper. The state of a scanned prefix is defined from the prefix: the thresholds b_1, b_2 as the least final values of increasing subsequences, the bands, and the interval stack by the legal moves of [definition 2.4](#), justified by [lemmas 2.2](#) and [2.8](#). The inductive invariant is the separation criterion of [lemma 2.11](#) and [corollary 4.4](#), from which two facts are derived, [proposition 2.10\(b\)](#) and (c): a letter from a deferred interval leaves no avoiding completion, and every complete word produced by legal moves avoids β_d . The completion counts then satisfy the literal recurrence by induction on the number of unread values, and the factorization (25) is proved directly from the equations of H by induction on the grade, without the abstract [theorem 2.17](#). The kernels are defined by their recurrences, so the theorems concern the recurrences as displayed. The restriction of the control sums is proved lossless, and it is also what makes the grade a well-founded order for (27) read as a definition.

The development has over ten thousand lines in twenty modules, contains no unproved statement, and each theorem depends only on Lean’s three standard axioms, as reported by the axiom sweep. The definitions were also exercised by kernel evaluation: the scans of the running example (4) reproduce the rows of [examples 2.12](#) and [4.2](#), kernel evaluation of the recurrences reproduces the first seven terms of (2), and compiled evaluation the first ten. Not formalized are the complexity statements of [theorem 1.1](#) and [corollary 7.4](#), the translation quotients of [section 7](#) and hence the program of [proposition 8.1](#), [theorem 2.17](#) in its general form, and the family for $d \geq 3$.

10. CONSEQUENCES AND OUTLOOK

Two patterns are *Wilf-equivalent* if they have equally many avoiders of every length. Reverse, complement, and inverse preserve all counting sequences. The dihedral orbit of 12453 has eight elements, and the prefix-reversal equivalence of Backelin–West–Xin [4] gives $12453 = 12 \oplus 231 \sim 21 \oplus 231 = 21453$, whose orbit supplies eight more. Thus the algorithm, bounds, and certified coefficients apply to all sixteen members of the length-five Wilf class represented by 12453.

Bóna determined the Stanley–Wilf limit of every member of the family [9]:

$$\lim_{n \rightarrow \infty} (a_n^{(d)})^{1/n} = (d - 1 + \sqrt{8})^2.$$

For the remainder of this section, put $a_n = a_n^{(2)} = |\text{Av}_n(12453)|$ and $\mu = (1 + \sqrt{8})^2 = 9 + 4\sqrt{2}$. Guided by the 2022 series analysis [11], the coefficients through $n = 100$ suggest the following refinement.

Conjecture 10.1. *There are constants $C, \kappa > 0$ and $h \in \mathbb{R}$ such that, as $n \rightarrow \infty$,*

$$a_n = C\mu^n e^{-\kappa n^{1/3}} n^{-17/4} \left(1 + hn^{-1/3} + O(n^{-2/3})\right). \quad (42)$$

The exponents $1/3$ and $-17/4$ were selected using only $a_0, \dots, a_{100}$. The parameters $\kappa, \log C, h$ were obtained by unweighted least squares on $70 \leq n \leq 100$ for the model $\log a_n = n \log \mu - \kappa n^{1/3} - \frac{17}{4} \log n + \log C + hn^{-1/3}$, and were fixed before $a_{101}, \dots, a_{150}$ were computed. The frozen values are $(\kappa, C, h) = (1.34550865, 0.82710184, 1.36800770)$ to the digits shown. On those fifty excluded coefficients, the maximum absolute difference between the observed and predicted values of $\log a_n$ is below $4.3 \cdot 10^{-7}$. See the supplement. From shorter series, the 2022 analysis estimated the stretched-exponential base $e^{-\kappa}$ (their μ_1) between 0.26 and 0.29 and the power-law exponent as -4.67 ± 0.1 [11]. The frozen fit gives $e^{-\kappa} \approx 0.260$, at the lower end of that range. Its exponent $-17/4$ differs from the 2022 estimate, but the two should be regarded as consistent rather than

in conflict. On $70 \leq n \leq 150$ the exponent $-17/4$ fits the model with one correction term far better than -4.67 does, but the preference shrinks by more than an order of magnitude once a second correction term $n^{-2/3}$ is admitted, and in fits with two correction terms a change in the stretched exponent $1/3$ is largely compensated by changes in the power-law exponent and in κ . The inferred power-law exponent is thus sensitive to the correction terms included, and the available coefficients do not determine it reliably. We record $-17/4$ as the simplest value consistent with the exact coefficients, not as a sharpening of the 2022 estimate. This supports [conjecture 10.1](#) but proves neither it nor any D-finiteness claim.

We close with four open problems. The first is to prove [conjecture 10.1](#). The second is to lower the arithmetic exponent 7 in the 12453 time bound. The third is to extend the certified series beyond $n = 150$. The fourth is to identify other length-five classes with protected tails, for instance among the classes generated by container machines [2], which by Theorem 1.1 of that paper are exactly the classes whose basis elements each begin with their largest entry.

DATA AND CODE AVAILABILITY

The code, coefficients, residues, reconstruction script, and computation reports used for [proposition 8.1](#), the sampler and heatmap tools of [section 8.1](#), the Permuta walkthrough with its exhaustive checks for length at most 8, and the Lean development of [section 9](#) are in the public repository

<https://github.com/ulfarsson/public-12453>

which is the supplement referred to throughout the paper. The code, data, and Lean development are released under the Apache License 2.0. The repository's MANIFEST.sha256 records individual file digests, and its README files give verification, reconstruction, and audit commands.

STATEMENT ON THE USE OF AI

Large language model assistants (Claude Opus and Claude Fable from Anthropic, and GPT Sol from OpenAI) were used throughout the preparation of this paper under the direction of the author: to draft and revise the text, to write and test the companion programs and run the computations reported in [section 8](#), and to develop the Lean formalization of [section 9](#). The mathematical content was specified and checked by the author. The computational claims are supported by the independent implementations and certificates described in [section 8](#), and the statements of [section 9](#) are verified by the Lean kernel. In addition, the Permuta walkthrough in the supplement checks every statement of the paper that has finite content exhaustively for all permutations of length at most 8, by comparing the paper's constructions and displayed formulas with brute-force computations from the definitions. These checks are written with the Permuta library [19], whose code predates the large language model assistants, and they do not use the implementations described in [section 8](#). The author takes full responsibility for the paper.

REFERENCES

1. Michael H. Albert, Christian Bean, Anders Claesson, Émile Nadeau, Jay Pantone, and Henning Ulfarsson, *Combinatorial exploration: an algorithmic framework for enumeration*, Mem. Amer. Math. Soc. **317** (2026), no. 1611, 109 pp.
2. Michael H. Albert, Cheyne Homberger, Jay Pantone, Nathaniel Shar, and Vincent Vatter, *Generating permutations with restricted containers*, J. Combin. Theory Ser. A **157** (2018), 205–232.
3. Michael H. Albert, Steve Linton, and Nik Ruškuc, *The insertion encoding of permutations*, Electron. J. Combin. **12** (2005), no. 1, Paper No. R47, 31 pp.
4. Jörgen Backelin, Julian West, and Guoce Xin, *Wilf-equivalence for singleton classes*, Adv. in Appl. Math. **38** (2007), 133–148.
5. Christian Bean, Émile Nadeau, Jay Pantone, and Henning Ulfarsson, *PermPAL: the Permutation Pattern Avoidance Library*, <https://permpal.com>, 2026, Accessed September 6, 2026.

6. Yonah Biers-Ariel, *Flexible schemes for pattern-avoiding permutations*, 2019, arXiv:1905.07283v2 [math.CO].
7. ———, *Julia program to compute terms of OEIS A116485*, https://oeis.org/A116485/a116485_1.txt, 2019, Accessed August 28, 2026.
8. Miklós Bóna, *Exact enumeration of 1342-avoiding permutations: a close link with labeled trees and planar maps*, J. Combin. Theory Ser. A **80** (1997), 257–272.
9. ———, *The limit of a Stanley–Wilf sequence is not always rational, and layered patterns beat monotone patterns*, J. Combin. Theory Ser. A **110** (2005), 223–235.
10. Noam Chomsky and Marcel P. Schützenberger, *The algebraic theory of context-free languages*, Computer Programming and Formal Systems (Paul Braffort and David Hirschberg, eds.), Studies in Logic and the Foundations of Mathematics, vol. 35, North-Holland, Amsterdam, 1963, pp. 118–161.
11. Nathan Clisby, Andrew R. Conway, Anthony J. Guttmann, and Yuma Inoue, *Classical length-5 pattern-avoiding permutations*, Electron. J. Combin. **29** (2022), no. 3, Paper No. P3.14, 63 pp.
12. Leonardo de Moura and Sebastian Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction – CADE 28, Lecture Notes in Computer Science, vol. 12699, Springer, 2021, pp. 625–635.
13. Philippe Flajolet and Robert Sedgewick, *Analytic combinatorics*, Cambridge University Press, Cambridge, 2009.
14. Scott Garrabrant and Igor Pak, *Permutation patterns are hard to count*, Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (Philadelphia, PA), SIAM, 2016, pp. 923–936.
15. John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, *Introduction to automata theory, languages, and computation*, second ed., Addison-Wesley, Boston, MA, 2001.
16. Adam Marcus and Gábor Tardos, *Excluded permutation matrices and the Stanley–Wilf conjecture*, J. Combin. Theory Ser. A **107** (2004), 153–160.
17. Albert Nijenhuis and Herbert S. Wilf, *Combinatorial algorithms for computers and calculators*, second ed., Academic Press, New York, 1978.
18. OEIS Foundation Inc., *A116485: permutations avoiding 12453*, <https://oeis.org/A116485>, 2026, Accessed August 28, 2026.
19. Permuta Triangle, *Permuta: a Python library for permutation patterns*, <https://github.com/PermutaTriangle/Permuta>, 2025, Version 2.3.1, released June 2025, accessed September 14, 2026.
20. Thomas Reps, Stefan Schwoon, Somesh Jha, and David Melski, *Weighted pushdown systems and their application to interprocedural dataflow analysis*, Sci. Comput. Program. **58** (2005), no. 1–2, 206–263.
21. Michael Sipser, *Introduction to the theory of computation*, third ed., Cengage Learning, Boston, MA, 2013.
22. The mathlib Community, *The Lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM, 2020, pp. 367–381.
23. Vincent Vatter, *Enumeration schemes for restricted permutations*, Combin. Probab. Comput. **17** (2008), no. 1, 137–159.
24. Doron Zeilberger, *Enumeration schemes and, more importantly, their automatic generation*, Ann. Comb. **2** (1998), 185–195.

DEPARTMENT OF COMPUTER SCIENCE, REYKJAVIK UNIVERSITY, MENNTAVEGUR 1, 102 REYKJAVIK, ICELAND
 Email address: henningu@ru.is